GitGithub

By Brian Kang
Last modified: 7/12/2026

1 Preface

What is Git? It is free open source version control system.
What is Version Control? It is a management of changes to code, and other collection of information.
CLI means Command Line Interface, Terminal, Git Bash, …
What is Github? It is a Git Server that offer free or private.

Not only collaborate with multiple developers, but it also makes 'working from home' more convenient.

2 Git Basic

2.1 Install

Visit "https://git-scm.com/download"

	Direction
Windows	Visit "https://git-scm.com/install/windows" and then click "Click here to download"
macOS	Type "git --version" and then follow-up direction
Linux	"sudo apt-get install git" on Debian/Ubuntu,

If you are using Windows, use "Git Bash" that provides bash on Windows Terminal, but also shows your current branch on prompt. Click Search Icon, and then type "Git Bash" to open.

2.2 First-Time Git Setup

After you install the Git, you have to configure your environment with "git config" command.
Add your name and email.

```
$ git config --global user.name "Brian Kang"
$ git config --global user.email briankang@example.com
```

Every Git commit uses this information.

Git has below three levels of configurations.

Location	Option	Area
`/etc/gitconfig`	`--system`	in the system-wide
`~/.gitconfig`	`--global`	which is specific to each user.
`.git/config`	`--local`	These values are specific to that single repository,

The default editor is “vi” or “vim”. If you want to change, for example VSCode, add below command.

```
$ git config --global core.editor "code --wait"
```

Windows and UNIX like OS are using different CRLF method. Use ‘true’ on Windows, input on Linux.

Windows	`core.autocrlf=true`
Linux or MacOs	`core.autocrlf=input`

Normally, Smart Git assigns default value, depend on your OS.

2.3 help

If you need help, type below. It will open the help on your default web Brower.

```
$ git help <verb>
$ git <verb> --help
$ man git-<verb>
```

Use “-h” option If you need a quick help. For example,

```
$ git add -h
```

2.4 Getting a Git Repository

To make a git repository, we can use two ways. At first, we will use ‘git init’.

1. Take a local directory, and turn it into a Git repository.

```
$ cd <your_working_directory>
$ git init
```

2. Clone an existing Git repository from elsewhere (Github)

```
$ cd <your_working_parent_directory>
$ git clone <URL>
$ cd <project directory>
```

The Github URL have https://github.com/<account>/<repository>.git , for example, https://github.com/vspauto/git-test.git. You can use “git clone <url> <your local directory name>” if you want to change local repository name.

Checking the Status of Your Files

To see the file status on WORKDIR and Stage Area, use “git status” command.

with git init	`bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)`
	`$ git status`
	`On branch master`
	
	`No commits yet`
	
	`nothing to commit (create/copy files and use "git add" to track)`

with git clone	`bongh@DESKTOP-INN3QB3 MINGW64 /e/github/tech-notes (main)`
	`$ git status`
	`On branch main`
	`Your branch is up to date with 'origin/main'.`
	
	`nothing to commit, working tree clean`

The default branch name is “`master`” if you used “git init”, and “`main`” if you used “git clone”.

2.5 Working directory, Stage Area and Repository.

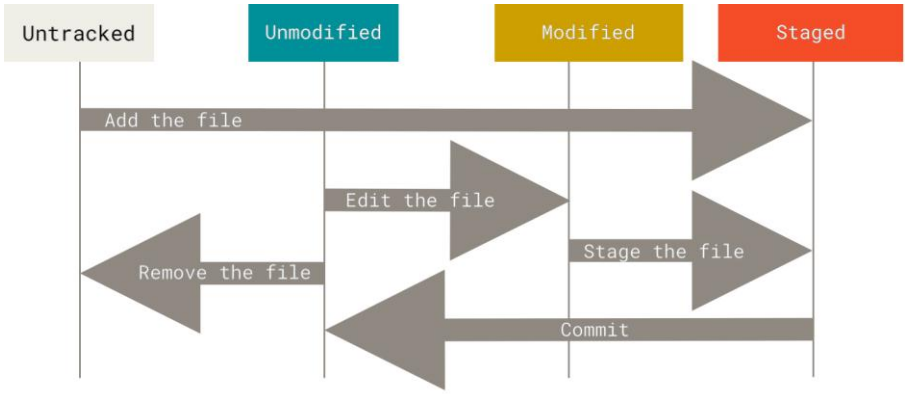
When you do ‘git init’ or ‘git clone’, it generates the “.git” sub-directory for stage area and local repository.

	Description
work dir	Working project directory that is tracking by Git.
stage area	Cache area between ‘work dir’ and ‘repository’.
repository	Your project snapshot area. Use commit command to move files on stage area to repository

If you are working with Github, we call it as ‘remote’, and call it as ‘origin’, default branch is ‘origin/main’.

2.6 Recording Changes to the Repository

Each file in your working directory can be in one of two states: *tracked* or *untracked*. Tracked files are files that were in the last snapshot, as well as any newly staged files; they can be *unmodified*, *modified*, or *staged*. In short, tracked files are files that Git knows about. Untracked files are any files in your working directory that were not in your last snapshot and are not in your staging area.



		Description
Untracked		New created file that does not added on stage area.
Tracked	Modified	Modified file
	Staged	Added file on stage area. It needs to commit to repository.
	Unmodified	File that is not modified after commit to repository

Tracking New Files

Create new file, for example README file, it is reported as untracked file, and Git suggests using “git add” command.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ echo "My Project" > README

$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README

nothing added to commit but untracked files present (use "git add" to track)

$ git status -s
```

```
?? README
```

Execute “git add” command to add file on Stage Area.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git add README

$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README

$ git status -s
A  README
```

To send files from Stage area to repository, use “git commit” command with “-m” option.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git commit -m 'Init Commit'
[master (root-commit) 6dc77cb] Init Commit
 1 file changed, 1 insertion(+)
 create mode 100644 README
```

Now, status is cleaned, and we can see the log history.

```
$ git status
On branch master
nothing to commit, working tree clean

$ git status -s

$ git log
commit 6dc77cb632214ec74dcf0a63ba389eb4d332cac0 (HEAD -> master)
Author: Brian Kang <bonghankang@gmail.com>
Date:   Fri Jun 26 16:55:26 2026 -0700

    Init Commit
```

Summary

create new file	We used echo command with ‘>’
git add <file> [file2…]	We can add several files, or all with “.”
git commit –m ‘commit message’	commit to repository with “-m” message
git log	to see the commit hostory
git status [-s]	to see status files on workdir and stage area
git diff [file]	report different between work dir and stage area

Modified Files

On the first commit state, we will modify README file, and then add on Stage Area.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ echo "Add 2nd Line" >> README
```

README file is updated, and Git suggests use ‘git add’ to update on stage area

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README
```

```
no changes added to commit (use "git add" and/or "git commit -a")

$ git status -s
 M README
```

If you want to see the different file between stage area and your work dir, use “git diff [file]” command.

```
$ git diff
diff --git a/README b/README
index 56266d3..5a0a7f3 100644
--- a/README
+++ b/README
@@ -1 +1,2 @@
 My Project
+Add 2nd Line
```

Add your modified file on Stage area.

```
$ git add README
```

```
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README

$ git status -s
M  README

$ git diff
```

If you want to see the different between the repository and stage area, use “git diff --staged [file]” command.

```
$ git diff --staged README
diff --git a/README b/README
index 56266d3..5a0a7f3 100644
--- a/README
+++ b/README
@@ -1 +1,2 @@
 My Project
+Add 2nd Line
```

If we commit, Git generates new commit and HEAD is moved to new generated commit.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git commit -m "Second commit"
[master 1706f1f] Second commit
 1 file changed, 1 insertion(+)
```

```
$ git log
commit 1706f1f01dff4f92de814c238d414821c2008258 (HEAD -> master)
Author: Brian Kang <bonghankang@gmail.com>
Date:   Fri Jun 26 21:09:32 2026 -0700

    Second commit

commit 6dc77cb632214ec74dcf0a63ba389eb4d332cac0
Author: Brian Kang <bonghankang@gmail.com>
Date:   Fri Jun 26 16:55:26 2026 -0700

    Init Commit
```

If file was added to Stage area and then if it is modified, we can use “-a” option on commit command that do “git add” and then “git commit”

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ echo "Add 3rd Line" >> README

$ git commit -a -m "Add 3rd Line"
 [master 1640f4c] Add 3rd Line
 1 file changed, 1 insertion(+)
```

Summary

modify file	We used echo command with “>>” to attach new line
git add <file> [files…]	Update stage area
git commit –m ‘commit message’	commit to repository. Git will generates new commit history
git commit –a –m ‘commit message’	We had been added file on stage area. We can do “git add” and “git commit” together. We can use “-am”.

git diff [file]	report different between stage area and work dir
git diff --staged [file]	report different between repository and stage area

Restore file (undoing)

If you accidently removed or modified file on stage area, `use "git restore --staged <file>..." to recover from repository.`

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git restore --staged README
```

If you accidently removed or modified file on work dir, use `"git restore <file>..." to recover from Stage area to work dir.`

```
$ git restore README
```

In this case, your modified file on WORKDIR cannot be recovered.

If you did commit file, and want to restore file from repository, you can use ‘git reset <commit id> <file>’ or ‘git checkout <commit id> <file>’ command. Below example will recover README file from the previous HEAD.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git checkout HEAD~1 README
Updated 1 path from 6dc77cb
```

The HEAD means the latest commit id, and HEAD~1 means the previous commit id.

Summary

git restore --staged <file>	Recover file from repository to stage area
git restore <file>	Recover file from stage area to work dir
git checkout HEAD <file>	Recover file from repository to stage area and work dir.
git reset HEAD <file>	Older, equivalent of ‘git restore --staged <file>’. ‘git reset’ is for the moving branch pointers, deleting commits, changing the staging area, and changing the working directory (depending on flags like `--soft`, `--mixed`, or `--hard`). Please use ‘git restore’

Delete file

If you want to delete file that already added on stage area and repository, we can use “rm” bash command, and “git add .” to update all changed on work dir to stage.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git status
On branch master
nothing to commit, working tree clean
```

```
$ rm README
$ git add .
$ git commit –m ‘remove README file’
```

The “git add .” command is to update all changes on work dir to stage area. So it will delete README file on stage area.

The more convent command that provided by Git is “git rm”. ‘git rm <file>’ command will delete file on Stage Area, but also file on WORKDIR.

```
$ git rm README
 $ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    README

 $ git status -s
D  README
$ git commit –m ‘remove README file’
```

If you accidently add file on stage area, `use "git rm --cached <file>..." to unstage.`

```
$ git rm --cached README
rm 'README'
```

It do not delete file on work dir, and file will be un-tracking state.

```
$ git status -s
?? README
$ ls
README
```

Summary

git rm <file>	Delete file on work dir and stage area
rm <file> git add .	Same as ‘git rm <file>’ command if ‘rm <file>’ was only changed on work dir
git rm --cached <file>	Delete file only on stage area

Rename File

We can move file. If use “git mv <src_file> <dst_file>”, file on stage area and workdir is moved.

```
$ git mv README.md README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README

$ git status -s
R  README -> README.txt
```

It is same as below commands

```
$ mv README.md README
$ git rm READ.md
```

```
$ git add README
```

Status with "-s" option

It provides short description for the status.
First column means Stage Area, and Second column means WORKDIR.

	Description
`$ git status -s` `?? README`	When you create new README file. Untracked file on Stage Area and WORKDIR
`$ git status -s` `A  README`	After "git add README" command. Added on stage area, and Unmodified on WORKDIR
`$ git status -s` `D  README`	After "`git rm README`". Deleted status on stage area, and removed on WORKDIR
`$ git status -s` ` M README`	When README file on WORKDIR is modified.
`$ git status -s` `M  README`	Modified README file is added on Stage area.
`$ git status -s` `R  README -> README.txt`	README file is moved to README.txt
`$ git status -s`	After "git commit" command. Unmodified both stage area and WORKDIR

.gitignore file

We can ignore some special files, for example, "*.o" file that is generated after compile C source code.

	Description
`*.[oa]`	Ignore any files ending in ".o" or ".a".
`!lib.a`	but do track lib.a, even though you're ignoring .a files above
`/TODO`	only ignore the TODO file in the current directory, not subdir/TODO
`build/`	ignore all files in any directory named build
`doc/*.txt`	ignore doc/notes.txt, but not doc/server/arch.txt
`doc/**/*.pdf`	ignore all .pdf files in the doc/ directory and any of its subdirectories

Rules
- Blank lines or lines starting with # are ignored.
- Standard glob patterns work, and will be applied recursively throughout the entire working tree.
- You can start patterns with a forward slash (/) to avoid recursively.
- You can end patterns with a forward slash (/) to specify a directory.
- You can negate a pattern by starting it with an exclamation point (!).

After you prepare .gitignore, add to stage area and commit to repository.

```
$ vi .gitignore
 *.[oa]
 !lib.a
 /TODO
 build/
 doc/*.txt
$ git add .gitignore
$ git commit –m 'add .gitignore'
```

Summary

Command	Description
`git init`	Initialize Git tracking and local repository
`git clone <url> [dir_name]`	Copy Git from URL to local directory
`git add <file> [file...]`	Add file on Stage Area. Use “.” for all.
`git commit [-m ‘string’]`	commit updates from Stage Area to repository
`git status [-s]`	Report status on WORKDIR and Stage Area
`git log`	Report commit history
`git restore --staged <file>`	Recover file from repository to Stage Area
`git restore <file>`	Recover file from Stage Area to WORKDIR
`git diff`	Compare WORKDIR and Stage Area
`git diff --staged`	Compare Stage Area and repository.
`git rm <file>`	Remove file on WORKDIR
`git rm --cached <file>`	Remove file on Stage Area
`git mv <old_fname> <new_fname>`	Rename file and then update on Stage area

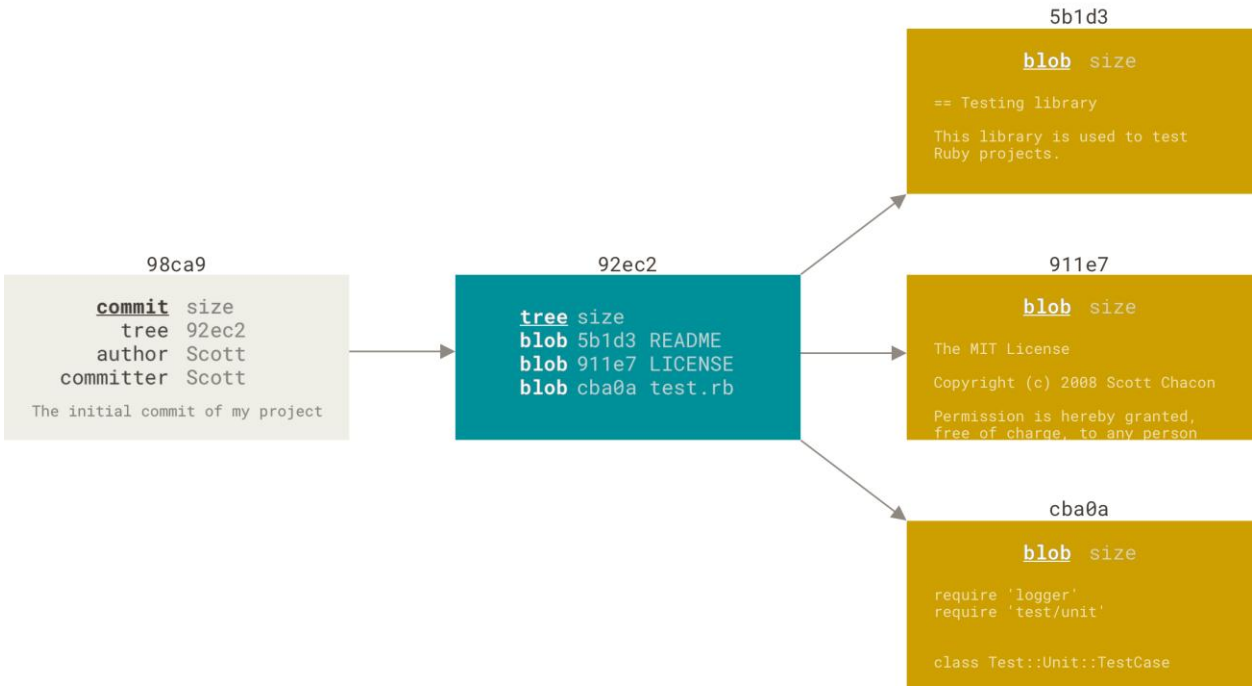
Note: “--cached" and “--staged" are meaning the Stage Area.

3 Commit History

When you create the commit by running ‘git commit’, Git checksums each subdirectory and stores them as a **tree object** in the Git repository. Git then creates a **commit object** that has the metadata and a pointer to the root project tree so it can re-create that snapshot when needed.

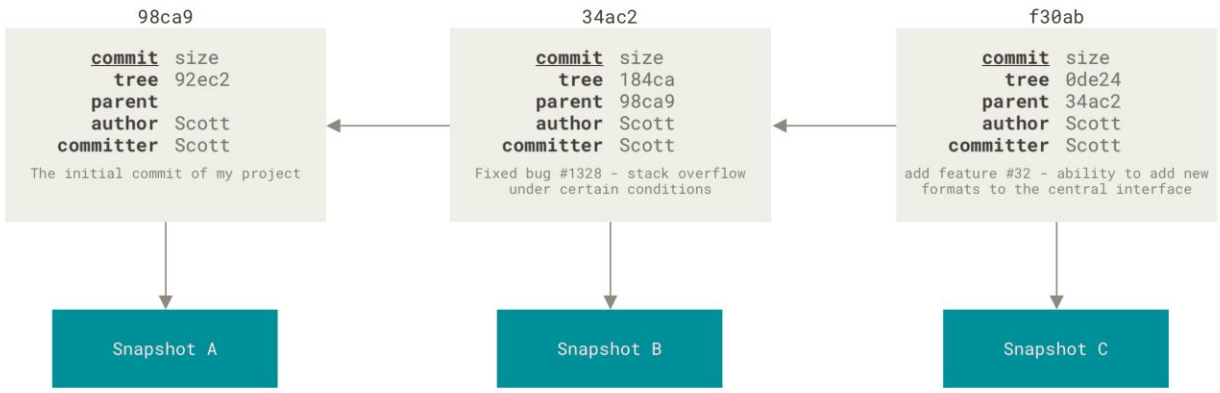
```
$ cd ~
$ mkdir git-test
$ cd git-test
$ git init
```

```
$ echo ‘ReadMe file’ > README
$ echo ‘test.rb file’ > test.rb
$ echo ‘LICENSE file’ > LICENSE
$ git add README test.rb LICENSE
$ git commit –m ‘The Initial commit of my project’
```

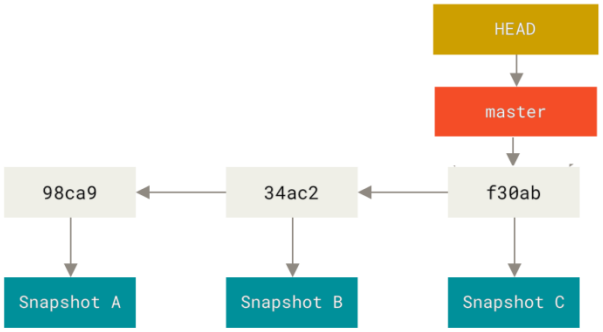


If you make some changes and commit again, the next commit stores a pointer to the commit that came immediately before it.

```
/* do modification */
$ git commit –a –m “Fixed bug #1328 – stack overflow ..”

/* do modification */
$ git commit –a –m “add feature #32 – ability to add ..”
```



The default branch name in Git is master. As you start making commits, you're given a master branch that points to the last commit you made. Every time you commit, the master branch pointer moves forward automatically. HEAD is indicating the latest commit id on current branch.



Git uses SHA-1 checksum with 40~41 characters, and we can use 7 first characters to denote the commit id. If you want to know the shortest prefix, use "**git rev-parse --short HEAD**" command.

3.1 Git Log

'git log' command provides the commit history.

Below options are more options that frequently used.

git log options	`git log [<options>] [<revision-range>] [[--] <path>…]`
git log	lists the commits made in that repository in reverse chronological order. This command lists each commit with its SHA-1 checksum, the author's name and email, the date written, and the commit message.
-p	or --patch, which shows the difference (the *patch* output) introduced in each

	commit.
-<number> -n<number>	Limit the number of commits to output. For example, “**git log -2**” will print last two commit histories.

--state	Generate a diffstat. For example,
	`README | 6 ++++++`
	`Rakefile | 23 +++++++++++++++++++++++`
	`lib/simplegit.rb | 25 +++++++++++++++++++++++++`
	`3 files changed, 54 insertions(+)`

--pretty[=<format>]
--format=<format>
	print the contents of the commit logs in a given format.
	Examples:
	`$ git log --pretty-oneline`
	`$ git log --pretty=format:"%h - %an, %ar : %s"`

--graph	Draw a text-based graphical representation of the commit history on the left hand side of the output.
	`$ git log --pretty=format:"%h %s" --graph`
	`* 2d3acf9 Ignore errors from SIGCHLD on trap`
	`* 5e3ee11 Merge branch 'master' of <your URL>`
	`|\`
	`| * 420eac9 Add method for getting the current branch`
	`* | 30e367c Timeout code and tests`
	`* | 5a09431 Add timeout protection to grit`
	`* | e1193f8 Support for heads with slashes in them`
	`|/`
	`* d6016bc Require time for xmlschema`
	`* 11d191e Merge branch ‘defunkt’ into local`

--since --until	the time-limiting options `$ git log --since=2.weeks`
--author	to filter on a specific author
--grep	lets you search for keywords in the commit messages.
-S	“pickaxe” option, which takes a string and shows only those commits that changed the number of occurrences of that string. For instance, if you wanted to find the last commit that added or removed a reference to a specific function, you could call: `$ git log -S function_name`

<revision-range>	A special notation "<*commit1*>..<*commit2*>" can be used as a short-hand for "^<commit1> <commit2>
	$ git log origin..HEAD
	$ git log HEAD ^origin

[[--] <path>...]	to pass a file path to git log as a filter.
	This is always the last option and is generally preceded by double dashes (--) to separate the paths from the options:
	`$ git log -- path/to/file`
	`$ git log README`
	
	list all the commits which are reachable from *foo* or *bar*, but not from *baz*.
	$ git log foo bar ^baz

The mixed example

```
$ git log --pretty="%h - %s" --author='Junio C Hamano' --since="2008-10-01" \
--before="2008-11-01" --no-merges -- t/
5610e3b - Fix testcase failure when extended attributes are in use
acd3b9e - Enhance hold_lock_file_for_{update,append}() API
f563754 - demonstrate breakage of detached checkout with symbolic link HEAD
d1a43f2 - reset --hard/read-tree --reset -u: remove unmerged new paths
```

```
51a94af - Fix "checkout --track -b newbranch" on detached HEAD
b0ad11e - pull: allow "git pull origin $something:$current_branch" into an unborn
branch
```

When you provides commit message, prepare message for below under bar area.

```
If applied to the codebase, this commit will ____
```

For example,

Upgrade the packages

Fix thread allocation

3.2 amend commit

We can add file after commit command.
As an example, if you commit and then realize you forgot to stage the changes in a file you wanted to add to this commit, you can do something like this:

```
$ git commit –m ‘Initial commit’
$ git add forgotten_file
$ git commit --amend
```

3.3 Tagging

Tagging is to make a release point(v1.0, v2.0 and so on).

It has two types. A lightweight tag is just a pointer to a specific commit. Annotated tags, however, are stored as full objects in the Git database.

	Description
git tag	show all tags
git tag –l “v1.*”	list option. show all tags start with “v1.”
git tag <tag name>	Add lightweight tag
git tag –a < tag name > -m <comment>	Annotated tags. It needs –m ‘comment’.
git tag –d <tag name >	Delete tag that is name with <tag name >

Add annotated tag ‘v1.0’.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git tag

$ git tag –a 'v1.0' –m ‘release v1.0’
```

Add lightweight tag ‘v1.1’.

```
$ echo "add new line" >> test_file.txt
$ git commit -am 'add new line on test_file.txt'
[master dbd9f9c] add new line on test_file.txt
 1 file changed, 1 insertion(+)

$ git tag v1.1
$ git tag
v1.0
v1.1
```

When you execute ‘git log’, it show additional tag information.

```
$ git log -2
commit dbd9f9ce60f4cca27313bc7ed178dc74c2c5fb22 (HEAD -> master, tag: v1.1)
Author: Brian Kang <bonghankang@gmail.com>
Date:   Thu Jul 2 20:07:26 2026 -0700

    add new line on test_file.txt

commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (tag: v1.0)
Merge: 99e57ca dab1f54
Author: Brian Kang <bonghankang@gmail.com>
Date:   Sun Jun 28 01:04:51 2026 -0700

    Solve conflict on main.c
```

When we use ‘git log’, both looks same, but if we use ‘git show’, annotated tag show additional information.

Annotated tags	`$ git show v1.0`
	`tag v1.0`
	`Tagger: Brian Kang <bonghankang@gmail.com>`
	`Date:   Thu Jul 2 19:59:16 2026 -0700`
	
	`add tag 1.0`
	
	`commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (tag: v1.0)`
	`Merge: 99e57ca dab1f54`
	`Author: Brian Kang <bonghankang@gmail.com>`
	`Date:   Sun Jun 28 01:04:51 2026 -0700`
	
	`    Solve conflict on main.c`
	
	`diff ...`

A lightweight tag	`$ git show v1.1`
	`commit dbd9f9ce60f4cca27313bc7ed178dc74c2c5fb22 (HEAD -> master, tag: v1.1)`
	`Author: Brian Kang <bonghankang@gmail.com>`
	`Date:   Thu Jul 2 20:07:26 2026 -0700`
	
	`    add new line on test_file.txt`
	
	`diff ...`

3.4 HEAD and commit id.

HEAD indicates the last commit that has ‘f30ab’ commit id on below diagram.
‘HEAD~” indicates the previous commit, ‘34ac2’.

To see the commit ID that related with HEAD, use “git rev-parse’ command.

```
$ git log --pretty=oneline -2
41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (HEAD -> master) Solve conflict on main.c
99e57ca1836812742e072dce5f45309354d90fa4 add printf on master branch
```

```
$ git rev-parse HEAD
41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5

$ git rev-parse HEAD~
99e57ca1836812742e072dce5f45309354d90fa4
$ git rev-parse HEAD^
99e57ca1836812742e072dce5f45309354d90fa4
$ git rev-parse HEAD^1
99e57ca1836812742e072dce5f45309354d90fa4
$ git rev-parse HEAD~1
99e57ca1836812742e072dce5f45309354d90fa4
```

We can use short commit id. To find out the short commit length, use below command.

```
$ git rev-parse --short HEAD
bac0710
```

It indicates 7 characters.

3.5 show command

‘git show’ provides similar information, but it based the object that includes the tag id. It is an **inspection** tool. It expands a single, specific Git object—most commonly a single commit—and shows exactly what changed inside it. It answers the question: *"What exactly happened in this specific commit?"*

command	Description
git show	show the latest commit on current branch with HEAD location
git show main	Shows the latest commit made on the main branch.
git show d3b4f12	Shows the author, date, message, and the exact lines added/removed in commit d3b4f12.
git show HEAD:README.md	Shows the entire contents of the README.md file exactly as it existed during the last commit, without showing a diff.

It provides similar commit history that includes the annotated tag.

```
$ git show v1.0
tag v1.0
Tagger: Brian Kang <bonghankang@gmail.com>
Date:   Thu Jul 2 19:59:16 2026 -0700

add tag 1.0

commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (HEAD -> master, tag: v1.0)
Merge: 99e57ca dab1f54
Author: Brian Kang <bonghankang@gmail.com>
Date:   Sun Jun 28 01:04:51 2026 -0700

    Solve conflict on main.c

diff --cc main.c
index 6161c72,9891a15..fbd9f35
--- a/main.c
+++ b/main.c
@@@ -2,6 -2,6 +2,7 @@@

  int main(int argc, char *argv[])
  {
```

```
 +    printf("Add on master branch\n");
+     printf("Add on testing branch\n");
      return 0;
--}
++}
```

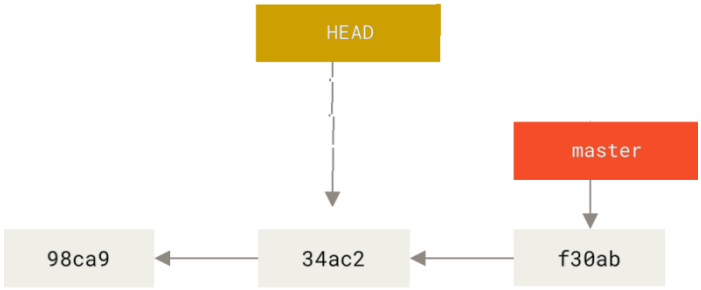
```
$ git show HEAD
$ git show 41d3bc9
```

3.6 checkout command

'git checkout <commit id>' provide you time slip. If you need the previous commit environment, we can use 'git commit HEAD~' command. It moves HEAD, and updates files on stage area and work dir.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git log -3 --pretty=oneline
commit bac0710cf52b43c6c971f9b2520428e6f9fc6489 (HEAD -> master)
commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5
commit 99e57ca1836812742e072dce5f45309354d90fa4
```

```
$ git checkout HEAD~
Note: switching to 'HEAD~'.
You are in 'detached HEAD' state.
..
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test ((41d3bc9...))
$
```




```
$ git log -3 --pretty=oneline
commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (HEAD)
commit 99e57ca1836812742e072dce5f45309354d90fa4
commit dab1f54f288753b8d2a3376aa4866ecf4e62d946 (testing)

$ git reflog
41d3bc9 (HEAD) HEAD@{0}: checkout: moving from master to HEAD~
bac0710 (master) HEAD@{1}: commit: test
41d3bc9 (HEAD) HEAD@{2}: checkout: moving from testing to master
dab1f54 (testing) HEAD@{3}: checkout: moving from master to testing
```

Now, you can validate something on commit id (41d3bc9).
Git said, you are in '**detached HEAD**' state, it means, HEAD is not on the last commit id.
And the Git Bash prompt indicates the commit id that HEAD currently linked.

To go-back to master, use 'git checkout master'.

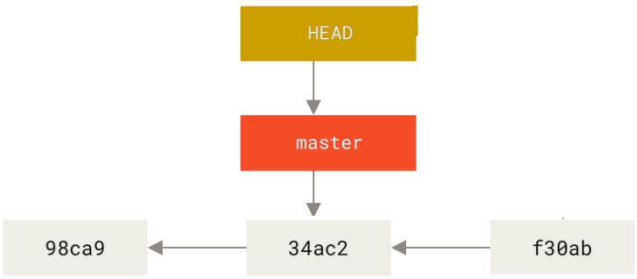
```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test ((41d3bc9...))
$ git checkout master
Previous HEAD position was 41d3bc9 Solve conflict on main.c
Switched to branch 'master'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git log -3 --pretty=oneline
commit bac0710cf52b43c6c971f9b2520428e6f9fc6489 (HEAD -> master)
```

```
commit 41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5
commit 99e57ca1836812742e072dce5f45309354d90fa4
```

3.7 RESET command

'git reset <commit id>' command move HEAD and branch link. So, it can clean up your last commit history.

```
$ git reset HEAD~

$ git log --pretty=oneline -3
41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 (HEAD -> master) Solve conflict on main.c
99e57ca1836812742e072dce5f45309354d90fa4 add printf on master branch
dab1f54f288753b8d2a3376aa4866ecf4e62d946 (testing) add printf on testing branch
```



The commit id (bac0710) becomes **Dangling Commit**. To see the dangling commit id, use 'git reflog' command.

```
$ git reflog
41d3bc9 (HEAD -> master) HEAD@{0}: reset: moving to HEAD~
bac0710 HEAD@{1}: checkout: moving from bac0710cf.. to master
bac0710 HEAD@{2}: checkout: moving from 41d3bc9cfe.. to bac0710
41d3bc9 (HEAD -> master) HEAD@{3}: checkout: moving from master to HEAD~
```

If you accidently executed it and want to go back, 'in this case (bac0710), we can use 'git reset HEAD@{1}' or 'git reset <commit id> (Before Git does garbage collection periodically).

```
$ git reset bac0710

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git log --pretty=oneline -3
bac0710cf52b43c6c971f9b2520428e6f9fc6489 (HEAD -> master) test
41d3bc9cfeaa82e7cdfff659228a42ddd6c8e4d5 Solve conflict on main.c
99e57ca1836812742e072dce5f45309354d90fa4 add printf on master branch
```

The reset has 3 options.

--soft	It only move HEAD on repository. The latest commits are removed.
--mixed	Default. (plus --soft), It restores files on stage area.
--hard	(plus --mixed), It restores files on work dir. In this case, we cannot recover file if it was modified.

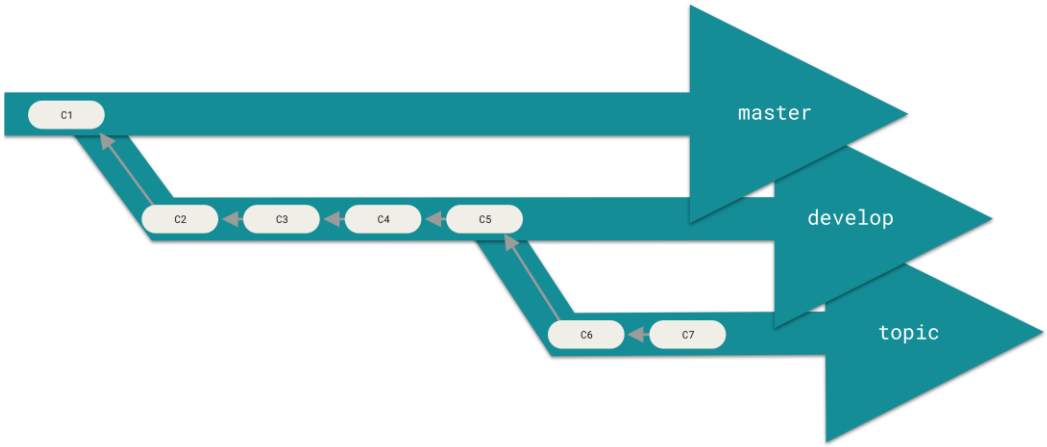
3.8 REVERT command

If we need a 'git reset' to reverse, but want to keep the history, we can use 'git revert <commit_id>' command. If we have a conflict, we need to merge manually, and then do 'git add .' and 'git revert --continue'

4 Branch

So far, we worked on master or main branch.
We can create new branch and work for other purpose, because Git uses a simple three-way merge, merging from one branch into another multiple times over a long period is generally easy to do.
Many Git developers have a workflow that embraces this approach, for example,

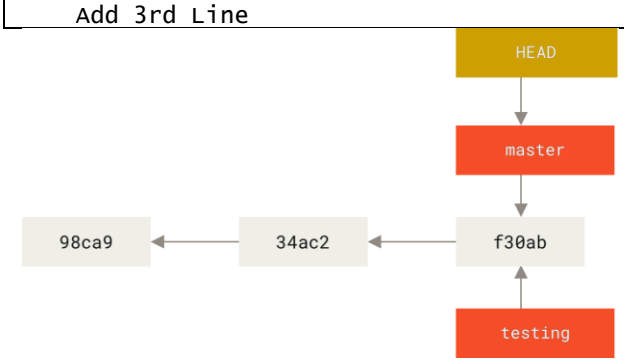
master	having only code that is entirely stable that possibly only code that has been or will be released.
develop next	they work from or use to test stability — it isn't necessarily always stable, but whenever it gets to a stable state, it can be merged into master.
topic	short-lived branches, (like your earlier iss53 branch) when they're ready, to make sure they pass all the tests and don't introduce bugs.
pu Proposed Update	that has integrated branches that may not be ready to go into the next or master branch.
Hotfix	To fix an emergency issue



4.1 Create Branch

To create new branch, use 'git branch <new branch>'

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git branch testing
$ git log
commit f30ab4cf66cab3edb6b9fcb379efc952b7fb0af6 (HEAD -> master, testing)
Author: Brian Kang <bonghankang@gmail.com>
Date:   Fri Jun 26 21:11:38 2026 -0700

    Add 3rd Line
```



To list branches, type “git branch”. Current branch is marked with (*).

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git branch
* master
  testing
```

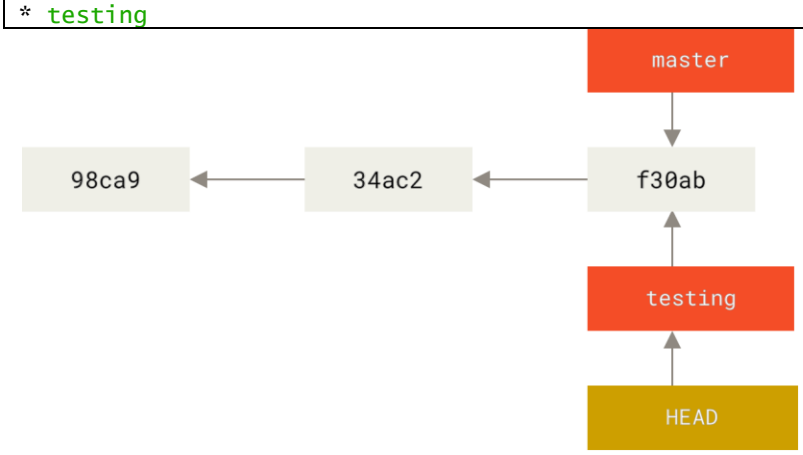
If you want to create branch and then switch, you can use ‘-b’ option. So below commands will have same result.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git branch testing
$ git checkout testing
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (testing)
```

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git checkout -b testing
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (testing)
```

4.2 switching Branch

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git checkout testing
Switched to branch 'testing'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (testing)
$ git branch
  master
* testing
```



‘git checkout’ command is used to move back/forward commit. So, modern Git provides ‘git switch <branch>’ command.

```
$ git switch master
$ git switch testing
```

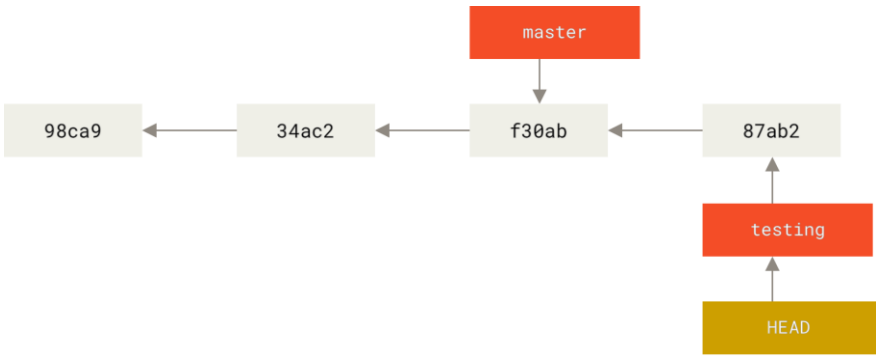
Use the ‘-d’ option when the temporary branch is no longer needed.

```
$ git checkout -b temp_branch
$ git branch
  master
* temp_branch
  testing
$ git checkout master
$ git branch -d temp_branch
```

4.3 Commit on branches

Commit on testing branch

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (testing)
$ echo "test text file" > test.txt
$ git add test.txt
$ git commit -a -m 'commit test.txt on testing branch'
[testing 87ab280] commit test.txt on testing branch
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt

$ git log --oneline --decorate
87ab280 (HEAD -> testing) commit test.txt on testing branch
f30ab4c (master) Add 3rd Line
34ac21f Second commit
98ca9cb Init Commit
```

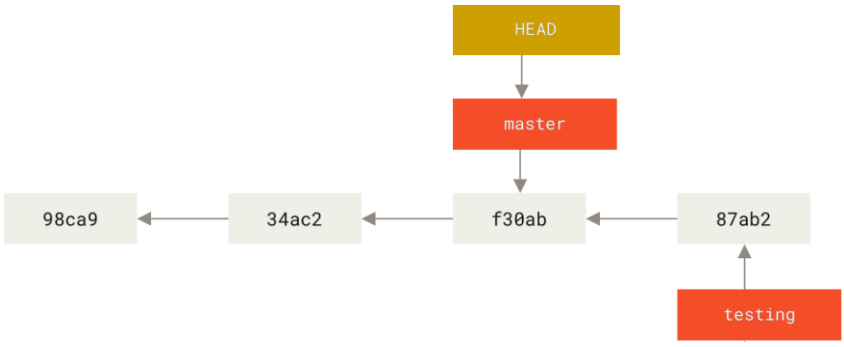


Move to master branch and commit.

At first, move back to master branch.

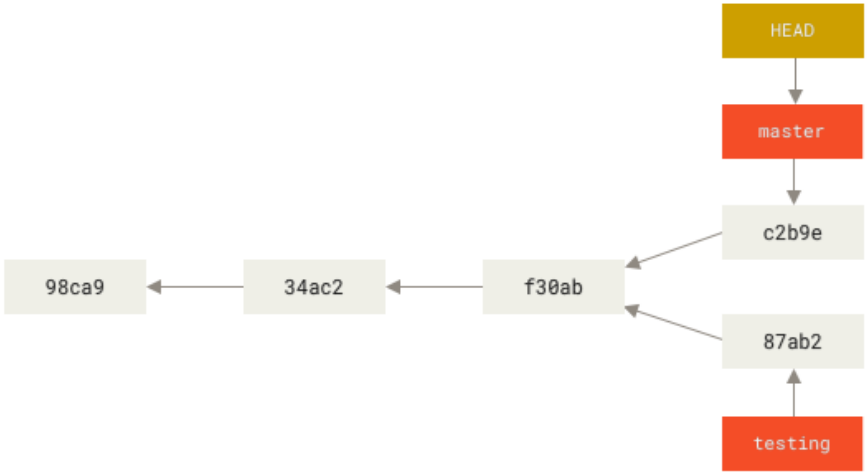
```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (testing)
$ git checkout master
Switched to branch 'master'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)

$ git branch
* master
  testing
```

HEAD is move back to master branch.



If we make one commit on master, it will have below picture.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ echo "test.txt file on master" > test.txt
$ git add .
$ git commit -m 'add test.txt on master'
$ git log --oneline --decorate --graph --all
* c2b9e (HEAD -> master) add test.txt on master
| * 87ab2 (testing) commit test.txt on testing branch
|/
* f30ab Add 3rd Line
* 34ac2 Second commit
* 98ca9 Init Commit
```

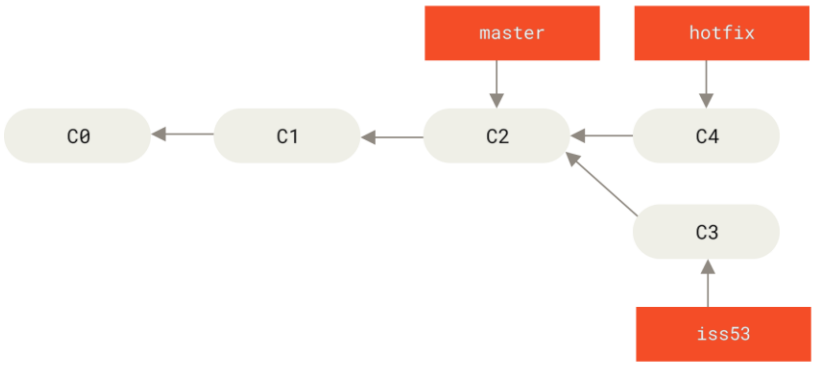


4.3 MERGE

Hotfix branch and Fast Forward Merge

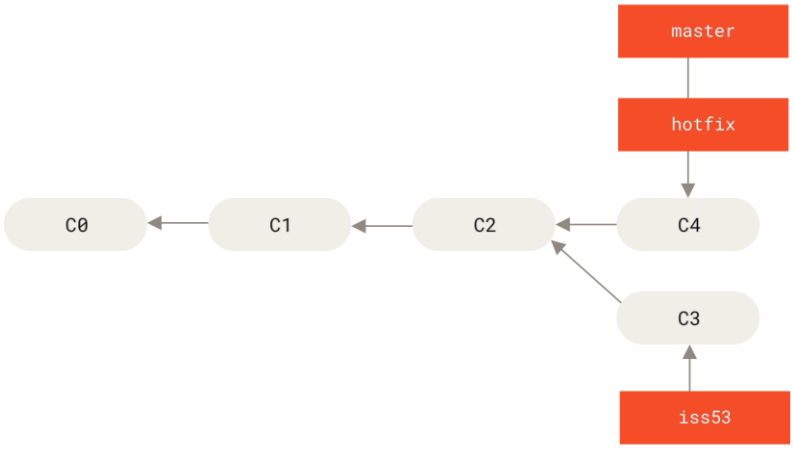
To make a fast forward merge, make a hotfix branch, and make one commit. Hotfix branch name is used when master has an issue, and need to fix it frequently.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git checkout -b hotfix
Switched to a new branch 'hotfix'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (hotfix)
$ echo "Hotfix on master" > hot.txt
$ git add hot.txt
$ git commit -m "Hotfix hot.txt file"
[hotfix 28a9543] Hotfix hot.txt file
 1 file changed, 1 insertion(+)
 create mode 100644 hot.txt

$ git log --oneline --decorate --graph --all
* 28a9543 (HEAD -> hotfix) Hotfix hot.txt file
| * 1abab80 (iss53) commit test.txt on testing branch
|/
* 1640f4c (master) Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit
```



We will merge hotfix branch. This merge is called as “Fast Forward” merge. (Because, it just moves the last master location from C2 to C4).

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (hotfix)
$ git switch master
Switched to branch 'master'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git merge hotfix
Updating 1640f4c..28a9543
Fast-forward
 hot.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 hot.txt

$ git log --oneline --decorate --graph --all
* 28a9543 (HEAD -> master, hotfix) Hotfix hot.txt file
| * 1abab80 (testing) commit test.txt on testing branch
|/
* 1640f4c Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit
```



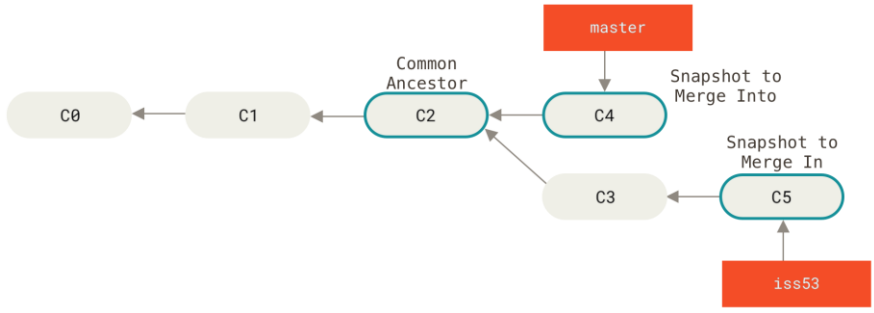
The ‘hotfix’ is not needed anymore, let’s delete it.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git branch -d hotfix
Deleted branch hotfix (was 28a9543).

$ git log --oneline --decorate --graph --all
* f0c3a75 (iss53) add test_file.txt on iss53
| * 28a9543 (HEAD -> master) Hotfix hot.txt file
|/
* 1640f4c Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit
```

Add one more commit on iss53

To make a 3 way merge environment, add two commits on iss53 branch.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git switch iss53
Switched to branch 'iss53'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (iss53)
$ echo "add 2nd line" >> test_file.txt

$ git commit -a -m "add 2nd line on test_file.txt"
[iss53 6b62951] add 2nd line on test_file.txt
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git log --oneline --decorate --graph --all
* 6b62951 (HEAD -> iss53) add 2nd line on test_file.txt
* f0c3a75 add test_file.txt on iss53
| * 28a9543 (master) Hotfix hot.txt file
|/
* 1640f4c Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit

$ git switch master
Switched to branch 'master'

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$
```

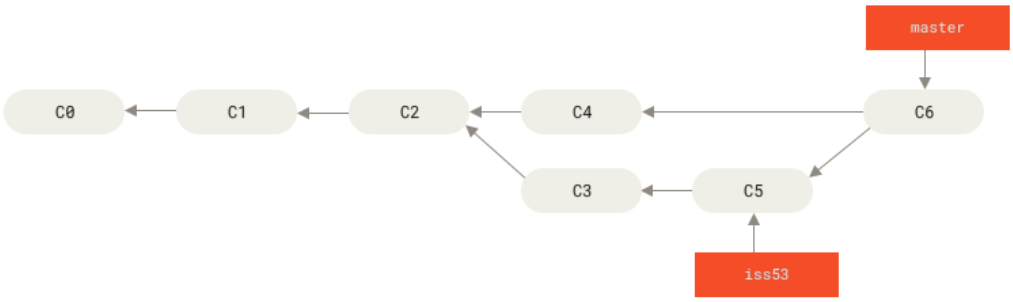


3-way merge

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git merge iss53
hint: Waiting for your editor to close the file... To read from stdin, append '-'
(e.g. 'echo Hello World | code -')
Merge made by the 'ort' strategy.
 test_file.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test_file.txt

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git log --oneline --decorate --graph --all
*   e85885a (HEAD -> master) Merge branch 'iss53'
|\
| * 6b62951 (iss53) add 2nd line on test_file.txt
| * f0c3a75 add test_file.txt on iss53
* | 28a9543 Hotfix hot.txt file
|/
* 1640f4c Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit

$ ls
```

```
README  hot.txt  test_file.txt
```

On three-way merge, it bases C2, and merge C4 and (C3..C5) and generates C6.



The test_file.txt is created on C3 and modified on C5 of iss53 branch, and then merged on master branch.

Remove unnecessary iss53 branch.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git branch -d iss53
Deleted branch iss53 (was 6b62951).
```

4.4 Conflict on Merge

Basic Merge Conflicts

Let's assume, we had main.c file, and add below one line on each branches.

On master branch	On testing branch

`#include <stdio.h>`
 	
`int main(int argc, char *argv[])`
`{`
`    printf("Add on testing branch\n");`
`    return 0;`
`}`
	`#include <stdio.h>`
	
	`int main(int argc, char *argv[])`
	`{`
	`    printf("Add on testing branch\n");`
	`    return 0;`
	`}`

When we executes 'git merge', it will report the merge conflict.

```
$ git switch master
$ git merge testing
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master|MERGING)
 $ git status
 $ git diff master testing
 $ git difftool
 $ git mergetool


$ cat main.c
#include <stdio.h>

int main(int argc, char *argv[])
{
<<<<<<< HEAD
    printf("Add on master branch\n");
=======
    printf("Add on testing branch\n");
>>>>>>> testing
    return 0;
```

```
}
```

This means the version in HEAD (your master branch) is the top part of that block (everything above the =======), while the version in your testing branch looks like everything in the bottom part. In order to resolve the conflict, you have to either choose one side or the other or merge the contents yourself.

To solve, open main.c, manually merge, and then commit with '-am' option.

```
$ vi main.c
```

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Add on testing branch\n");
    printf("Add on testing branch\n");
    return 0;
}
```

```
$ git commit -am 'Solve conflict on main.c'
[master 41d3bc9] Solve conflict on main.c

bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master)
$ git log --oneline --decorate --graph --all
*   41d3bc9 (HEAD -> master) Solve conflict on main.c
|\
| * dab1f54 (testing) add printf on testing branch
* | 99e57ca add printf on master branch
|/
* 6063f42 add main.c
*   e85885a Merge branch 'iss53'
|\
| * 6b62951 add 2nd line on test_file.txt
| * f0c3a75 add test_file.txt on iss53
* | 28a9543 Hotfix hot.txt file
|/
* 1640f4c Add 3rd Line
* 1706f1f Second commit
* 6dc77cb Init Commit
```

Done, we solved conflict!

With merge tool, meld

The merge tools frequently used by developers are as follows:

Tool	Description
KDiff3	A free, open-source cross-platform tool that integrates easily with Git and Subversion. It is well-regarded for its smart algorithm and manual conflict resolution controls.
Meld	A free and open-source visual diff and merge tool popular among Linux and Windows users for its straightforward 2- and 3-way comparison
WinMerge	Highly popular, free, and open-source visual file and folder differencing and merging tool for Windows.
P4Merge	highly recommended for complex 3-way merges. It offers a 4-pane view (Yours, Theirs, Base, and Merged).
VSCode	Best IDE.

To use the "meld" for merge, you should install meld and need to edit "~/gitconfig" file. For example,

```
bongh@DESKTOP-INN3QB3 MINGW64 ~
$ cat .gitconfig
[user]
        email = brian.kang@example.com
        name = Brian Kang
```

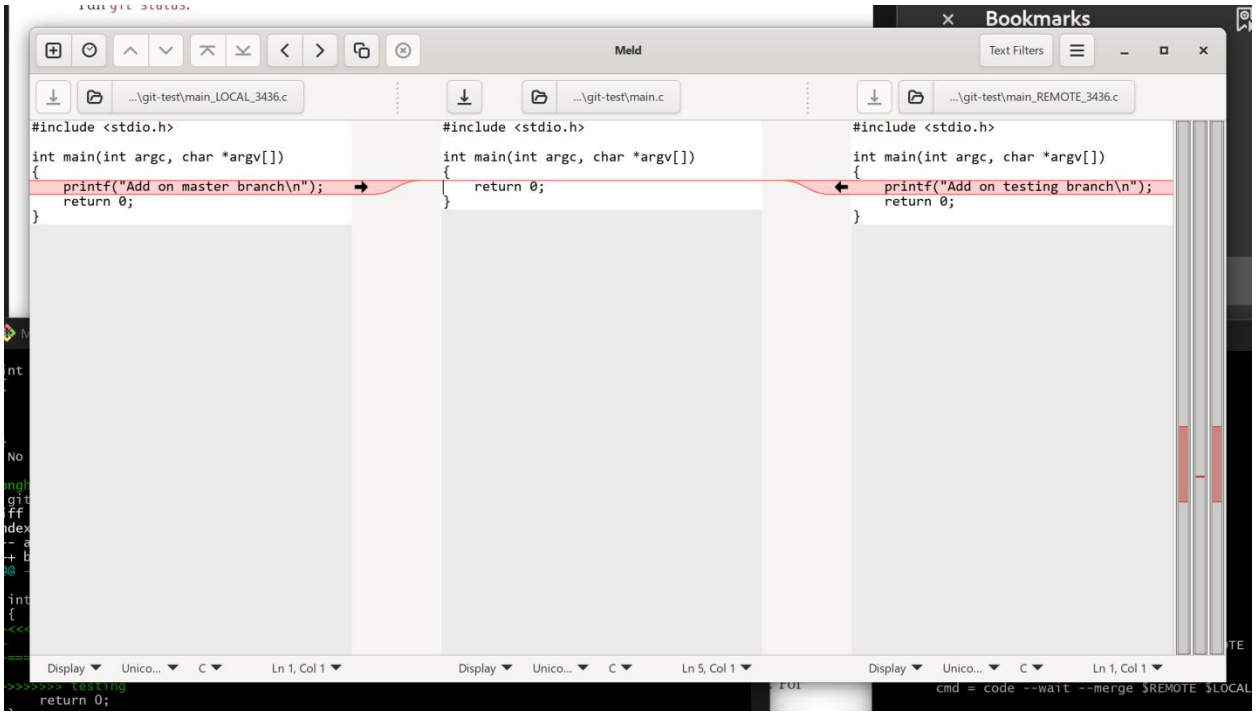
```
[core]
        autocrlf = true
        editor = code --wait

#-----------------------------
# select default tool
#-----------------------------
[diff]
    # tool = vscode
    tool = meld

[merge]
    # tool = vscode
    tool = meld

#-----------------------------
# common
#-----------------------------
[difftool]
    prompt = false

[mergetool]
    prompt = false
    keepBackup = false

#-----------------------------------
# VSCode
#-----------------------------------
[difftool "vscode"]
        cmd = code --wait --diff $LOCAL $REMOTE

[mergetool "vscode"]
        cmd = code --wait --merge $REMOTE $LOCAL $BASE $MERGED

#-----------------------------------
# Meld
#-----------------------------------
[difftool "meld"]
    cmd = meld "$LOCAL" "$REMOTE"

[mergetool "meld"]
    cmd = meld "$LOCAL" "$BASE" "$REMOTE" --output "$MERGED"
```

The $LOCAL, $BASE, $REMOTE are replaced with below files.

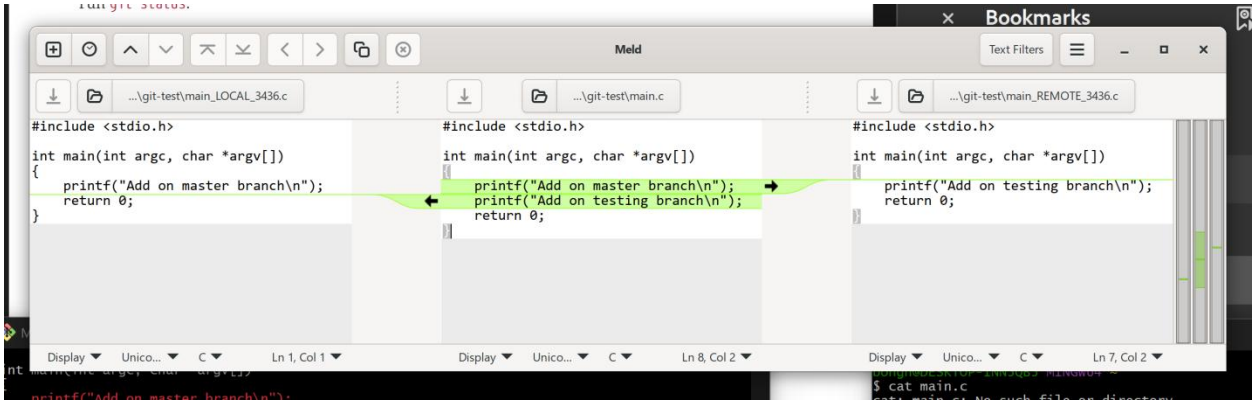
```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master|MERGING)
$ ls -al
total 44
drwxr-xr-x 1 bongh 197609   0 Jun 28 01:00 ./
drwxr-xr-x 1 bongh 197609   0 Jun 28 00:47 ../
drwxr-xr-x 1 bongh 197609   0 Jun 28 00:45 .git/
-rw-r--r-- 1 bongh 197609 156 Jun 28 01:00 main.c
-rw-r--r-- 1 bongh 197609 194 Jun 28 00:49 main_BACKUP_3436.c
-rw-r--r-- 1 bongh 197609  75 Jun 28 00:49 main_BASE_3436.c
-rw-r--r-- 1 bongh 197609 114 Jun 28 00:49 main_LOCAL_3436.c
-rw-r--r-- 1 bongh 197609 115 Jun 28 00:49 main_REMOTE_3436.c
...other files...
```

Execute ‘git mergetool’, it will execute linked ‘meld’ with 3 panels.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/git-test (master|MERGING)
$ git mergetool
Merging:
main.c

Normal merge conflict for 'main.c':
  {local}: modified file
  {remote}: modified file
```



Change as below, and then save.



4.5 Branch Management

To see all branches, use “git branch” command.

```
$ git branch
  iss53
* master
  testing
```

Notice the * character that prefixes the master branch: it indicates the branch that you currently have checked out.

To see the last commit on each branch, you can run git branch -v:

```
$ git branch -v
  iss53 93b412c Fix javascript issue
* master 7a98805 Merge branch 'iss53'
```

```
  testing 782fd34 Add scott to the author list in the readme
```

We can use '--merged' option to find out the merged branch, and use "--no-merged" to find out the branch that need to merge.

```
$ git branch --merged
  iss53
* master
```

```
$ git branch --no-merged
  testing
```

If we try to delete the testing branch that is not fully merged, it can be failed.

```
$ git branch -d testing
error: The branch 'testing' is not fully merged.
If you are sure you want to delete it, run 'git branch -D testing'.
```

To see the files that is not merged on your branch (ex: testing), checkout to your branch and use below command.

```
$ git checkout testing
$ git branch --no-merged master
  topicA
  featureB
```

Changing a branch name

We can rename branch name with 'git branch –m'. (long name is '--move')

```
$ git branch --move bad-branch-name corrected-branch-name
```

Note: Do not rename branches that are still in use by other collaborators.

If you started with 'git init' and then used 'git add remote <url>' to link Github, we need to rename branch name from 'master' to 'main'. Use below example. We will learn more detail later.

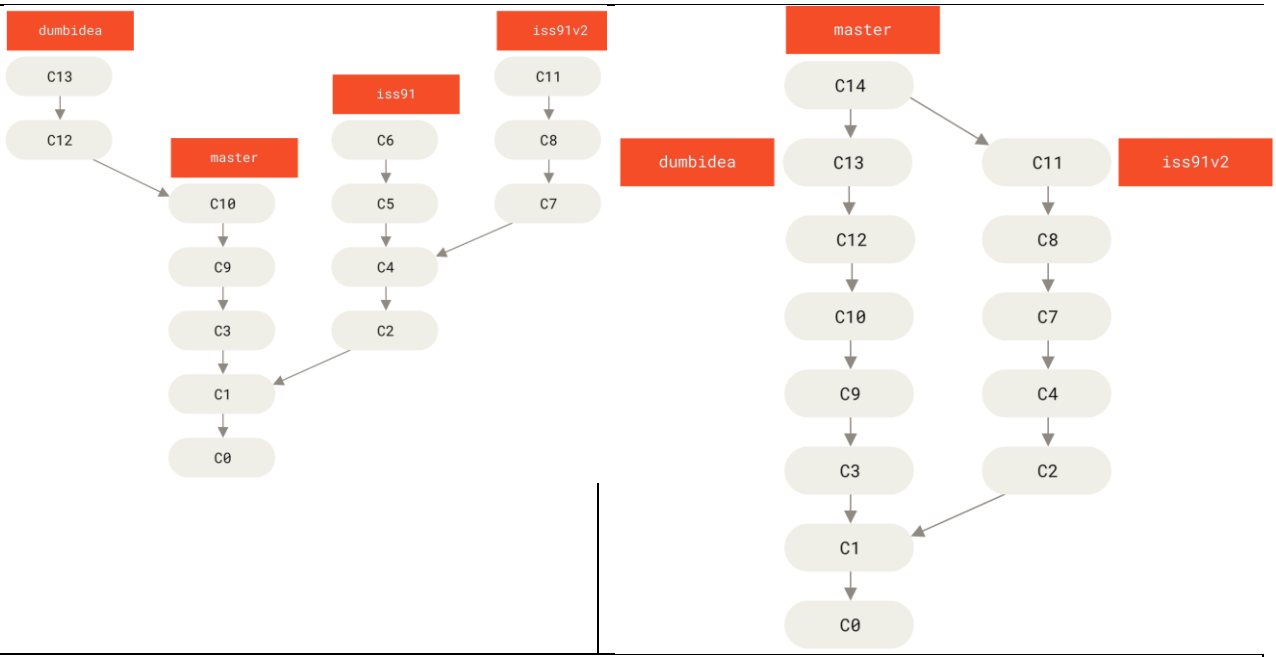
```
$ git branch --move master main
$ git push --set-upstream origin main
$ git branch --all
* main
  remotes/origin/HEAD -> origin/master
  remotes/origin/main
  remotes/origin/master
$ git push origin --delete master
```

Note: it will break the integrations, services, helper utilities and build/release script that your repository uses.

4.6 Branch workflow

Below show some topic branches, and how to merge it.

from	to

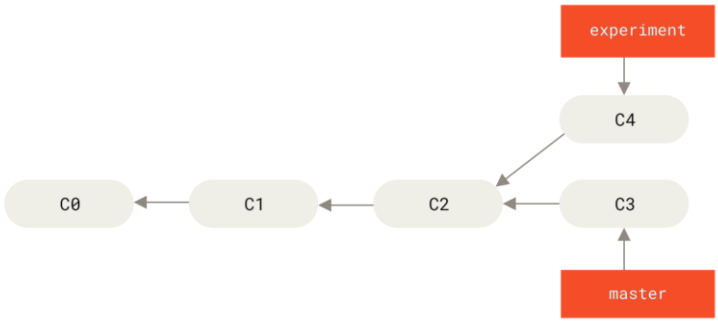



```
$ git checkout master
$ git merge dumbidea
$ git merge iss91v2
$ git branch -d iss91
/* $ git branch –d dumbidea */
```

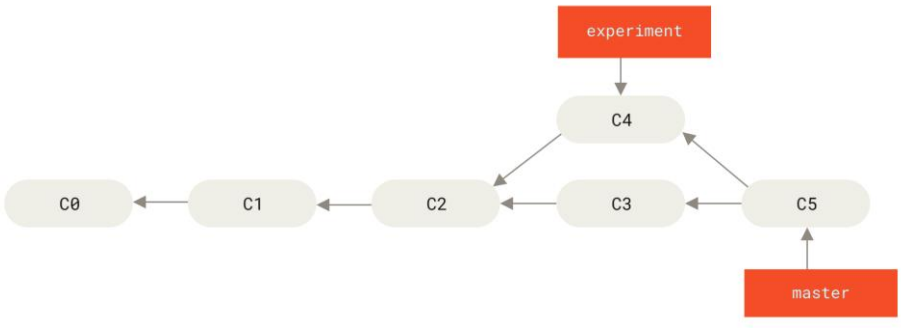
4.7 Rebasing

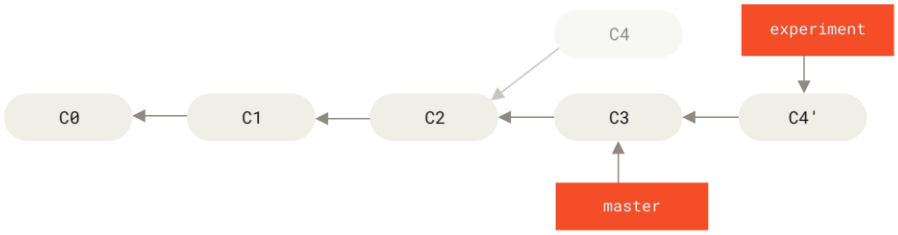
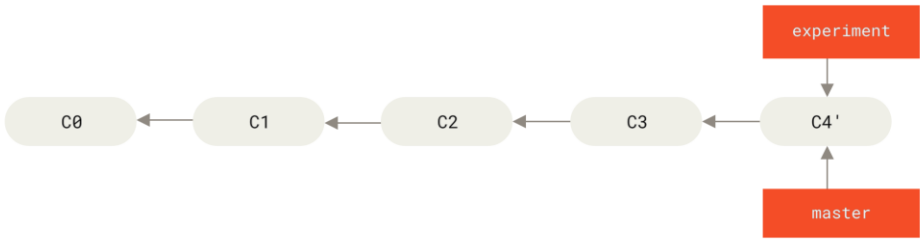
It is to make a linear commit history.
Do not use on released repository, because C4 commit information is changed as C4'.

Base



with 3-way



with rebase	$ git checkout experiment
	$ git rebase master
	First, rewinding head to replay your work on top of it...
	Applying: added staged command
	 
	
	$ git checkout master
	$ git merge experiment
	 

4.8 stash

Stash is a stack to save your un-commit files. It temporarily shelves (stashes) changes you've made to your working directory so you can work on something else, and then come back and re-apply them later. This is incredibly useful when you are in the middle of a code change, your working directory is messy, and you suddenly need to switch branches to fix an urgent bug without committing unfinished work.

git stash	Save modified and staged changes. OR to give it a memorable description: $ git stash save "Working on feature X, almost done"
git stash -u	By default, `git stash` only tracks **staged** and **unstaged** changes to tracked files. It ignores untracked or ignored files. To include **untracked files** (new files not yet added to Git), use '-u' option.

git stash list	list stack-order of stashed file changes. for example
	`$ git stash list`
	`stash@{0}: On main: Working on feature X, almost done`
	`stash@{1}: On develop: temporary WIP`

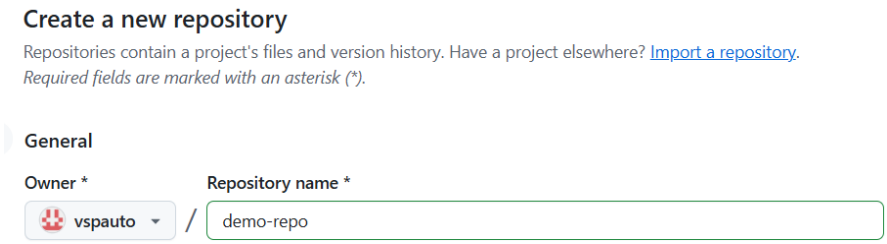
git stash pop	When you are ready to resume work, restore your stashed changes. If you have several stash items, you can select the item. For example, $ git stash pop stash@{1}
git stash drop	discard the changes from top of stash stack

5 Remote

5.1 Prepare repository on Github

Visit https://github.com, sign-up.

Select  on top/left, and select new repository.

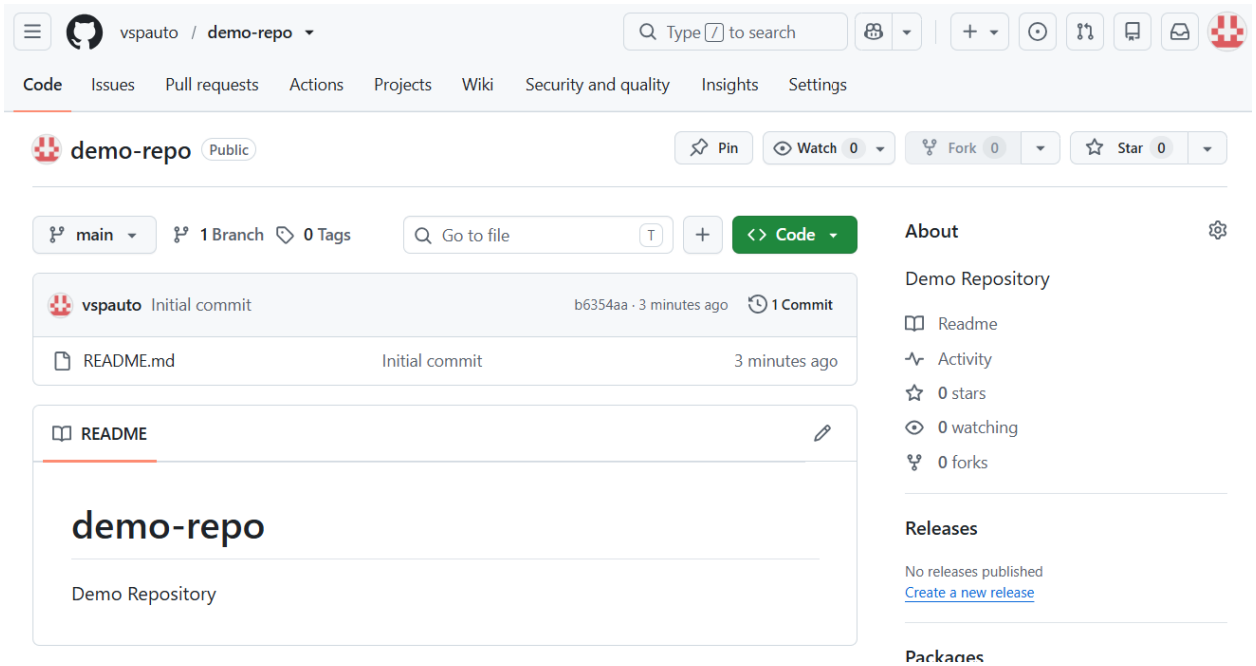


Select below items, and then click  .

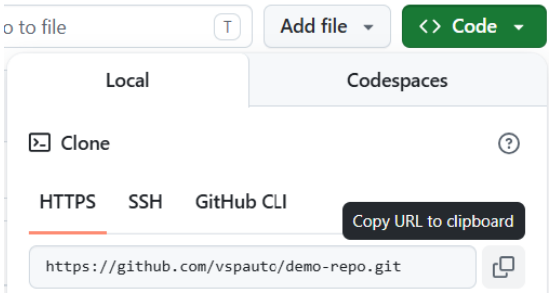
Item	
Owner	In my case, vspauto
Repository name	‘demo-repo’
Description	It will appear on ‘github.com/vspauto’, and used on README.md
visibility	Select public
Add README	Select ON
.gitignore	No .gitignore. We will change it later
License	Select No License

Note: sometimes, others suggest turn off the ‘Add README’ to solve the common ancestor" issue.

Github generates “github.com/vspauto/demo-repo” page.



Select “<> code” and copy <url> to use on the next chapters. In this case, it is ‘https://github.com/vspauto/demo-repo.git’.



5.2 Link remote repository

To use a remote repository like GitHub, we use the names 'main' branch and 'origin'.

	Description
master	Default branch name when using only a local repository
main	Default branch name when connect to Remote repository
origin	Default name for Remote repository.

We have 3 methods to link remote repository. On the next examples, let’s assume you have “https://github.com/vspauto/demo-repo.git” and have a write right.

method 1. ‘git clone’ on new directory

On the below example, we will clone the remote repository on “~/demo-repo”.

```
$ cd ~
$ git clone https://github.com/vspauto/demo-repo.git
Cloning into 'demo-repo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

$ cd demo-repo

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
$
```

method 2. ‘git remote add’ on new directory

To link remote (Github) repository to local, we will use ‘git remote add’ and ‘git push’ command.

	Description
git remote add <remote_name> <url>	Add remote on <remote_name>. Default is origin.
git pull <remote_name> <local_branch>	Fetch and merge <remote_name> to <local_branch>

The default <remote_name> is ‘origin, and sometimes people denotes it as <name>, or <short name>.
The default branch ‘main’ on remote repository is represented as ‘origin/main’ on the local.

If you start your project locally with ‘master’, at first, use “git branch –m master main’ command to rename it as main. And then execute ‘git remote add’, and ‘git pull’ commands.

```
$ cd ~
```

```
$ mkdir demo-repo1
$ cd demo-repo1
$ git init
Initialized empty Git repository in C:/Users/bongh/demo-repo1/.git/

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (master)

$ git branch -m master main

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (main)
$ git remote add origin https://github.com/vspauto/demo-repo.git

$ git pull origin main
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 11 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (11/11), 1.60 KiB | 17.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
```

On the upper example, we used “-m” (--move) option to move master to main before we do ‘git commit’ and ‘git remote add’.

If you want to use remote repository with ‘main’ branch, add below to make ‘main’ as default.

```
$ git config --global init.defaultBranch main
```

When you run ‘git init’, it will start with ‘main’, not ‘master’

method 3. Connect an existing local ‘master’ to remote repository

If you did commit on your local repository on ‘master, you will have ‘main vs. master’ issue.
On ‘master’, if you push to Github, Github will generate new branch with ‘master’ that we do not want.
If you did commit on master, and then rename it as ‘main’, the mismatched history issue will be happened.

In the below example, we did one commit. When we check the branch, it already has ‘master’ or ‘main’.

```
$ cd ~
$ mkdir demo-repo2
$ cd demo-repo2
$ git init
Initialized empty Git repository in C:/Users/bongh/demo-repo2/.git/

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (master)
$ git branch

$ echo “file on master” > test.txt
$ git add test.txt
[master (root-commit) 09507b0] add test.txt on master
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
$ git branch
* master

$ git branch -m master main

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (main)
$ git branch
* main

$ git remote add origin https://github.com/vspauto/demo-repo.git
```

When you execute ‘git pull’, the fetch step is working, but the merge step is failed.

```
$ git pull origin main
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 12 (delta 0), reused 10 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (12/12), 1.76 KiB | 14.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
fatal: refusing to merge unrelated histories
```

```
$ git pull origin main
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
fatal: refusing to merge unrelated histories
```

Git is refusing to merge.

This happens because your local repository and your GitHub repository have entirely separate timelines (histories). Since they don't share a common "ancestor" commit, Git panics and protects you from accidentally overwriting code.
This almost always happens when you initialize a repository on GitHub (with a README or .gitignore), initialize a separate repository locally, and then try to pull or push them together.

To solve this issue, you need to tell Git to ignore the unrelated histories just this once using a special flag.

```
$ git pull origin main --allow-unrelated-histories
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
hint: Waiting for your editor to close the file... To read from stdin, append '-'
(e.g. 'echo Hello World | code -')
 Merge branch 'main' of https://github.com/vspauto/demo-repo
Merge made by the 'ort' strategy.
 README.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 README.md

$ ls
README.md  test.txt
```

In this case, ‘README.md’ is on remote and local, but ‘test.txt’ is only on the local repository.
To update ‘test.txt’ on remote, use ‘git push’ command.

```
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 532 bytes | 106.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/vspauto/demo-repo.git
   196d873..ccdb759  main -> main
branch 'main' set up to track 'origin/main'.
```

And click “reload” button on Brower. It will show ‘README.md’ and ‘test.txt’ file on your Brower.

Summary

git clone <url> cd <project_name>	Clone
git remote add origin <url> git push origin <branch>	Add remote on origin. Fetch and merge on local branch.

5.3 remote commands

On this chapter, we will use below commands.

	Description
git remote	list remote
git remote add origin <url>	Link remote <url> at origin. Default <remote_name> is origin.
git remote -v	--verbose, it show linked <url> that Git has stored for the short name to be used when reading and writing to that remote
git fetch <name>	Download from remote to local FETCH_HEAD
git merge [<name>/<branch>]	merge fetched files. 'git merge' means 'git origin/main' or 'git merge FETCH_HEAD'
git pull <name> <branch>	do 'git fetch' and 'git merge'
git push –u <url> <branch>	-u means, "set-upstream", set upstream for git pull/status. git push –u origin main
git push	use up-stream
git remote delete <name>	Delete remote branch

Add Remote

Assume the remote repository <url> is "http://github.com/vspauto/demo-repo.git'.

```
$ git remote add origin http://github.com/vspauto/demo-repo.git
```

Download from Remote and merge on Local Repository

Use 'git pull origin main' command, or use below two commands.

```
$ git fetch origin main
$ git merge origin/main
```

We can also use 'git merge' or 'git merge FETCH_HEAD'.
These two commands also used to sync the remote and the local, before you push (upload) your modification.

List Remote

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (main)
$ git remote
origin

$ git remote -v
origin  https://github.com/vspauto/demo-repo.git (fetch)
origin  https://github.com/vspauto/demo-repo.git (push)

$ git ls-remote origin
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        HEAD
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/main
```

Upload

Create main.c on your local repository.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (main)
$ code main.c
```

```
#include <stdio.h>

int main(int argc,char *argv[])
{
    printf("Hello World\n");
    return 0;
}
```

```
$ git add main.c
$ git commit -m 'add main.c'
```

Check before you do ‘git pull’.

```
$ git pull origin main
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
Already up to date.
```

Upload file.

```
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 393 bytes | 98.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/vspauto/demo-repo.git
   ccdb759..f9f3f47  main -> main
branch 'main' set up to track 'origin/main'.
```

And click “reload” button on Brower. It will show new ‘main.c’ file on your Brower.
‘-u’ option on ‘git push’ is a ‘--**set-upstream**’ that links your local main branch to the remote origin/main branch, so in the future, you only need to type ‘git pull’, git ‘fetch’ or ‘git push’ when working on it.

If you push on “~/demo-repo2’, let’s sync on “~/demo-repo”, do sync.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
$ git remote -v
origin  https://github.com/vspauto/demo-repo.git (fetch)
origin  https://github.com/vspauto/demo-repo.git (push)

$ ls
README.md
```

```
$ git fetch origin main
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 512 bytes | 11.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
   196d873..ccdb759  main       -> origin/main

$ git merge
Updating 196d873..ccdb759
Fast-forward
 test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt

$ ls
```

```
README.md  main.c
```

'git log' provides 'origin/main' that is related with remote.

```
$ git log -4
commit ccdb7598f844a3d99e16a0591d49d4d3a24c3d64 (HEAD -> main, origin/main,
origin/HEAD)
Merge: 09507b0 196d873
Author: Brian Kang <bonghankang@gmail.com>
Date:   Thu Jul 9 14:19:13 2026 -0700

    Merge branch 'main' of https://github.com/vspauto/demo-repo

commit 09507b083453b043d7583a482222a73aca7abf60
Author: Brian Kang <bonghankang@gmail.com>
Date:   Thu Jul 9 13:57:40 2026 -0700

    add test.txt on master

commit 196d873dbdf4bf9250748c1695901b9e16a5b9be
Author: Brian Kang <bonghankang@gmail.com>
Date:   Thu Jul 9 11:35:29 2026 -0700

    remove local.txt test.txt
```

5.4 Remote Branch

Remote references are references (pointers) in your remote repositories, including branches, tags, and so on. Use "git ls-remote <remote>" or "git remote show <remote>"

```
$ git ls-remote origin
From https://github.com/vspauto/tech-notes
83209c95a28083be04210bc72fe34c306f504a4f        HEAD
83209c95a28083be04210bc72fe34c306f504a4f        refs/heads/main
```

```
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/vspauto/tech-notes
  Push  URL: https://github.com/vspauto/tech-notes
  HEAD branch: main
  Remote branch:
    main tracked
  Local branch configured for 'git pull':
    main merges with remote main
  Local ref configured for 'git push':
    main pushes to main (up to date)
```

Remote-tracking branch names take the form **<remote>/<branch>**. For instance, if you wanted to see what the **main** branch on your **origin** remote looked like as of the last time you communicated with it, you would check the **origin/main** branch.

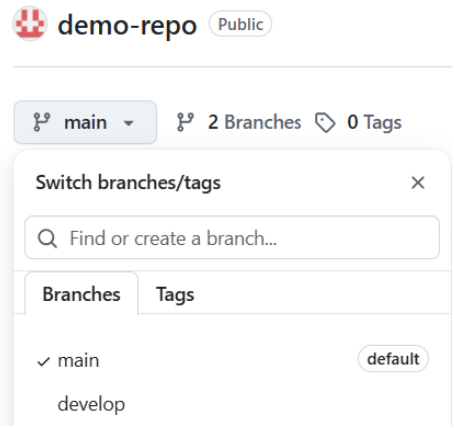
Create branch

If you want to create new branch, for example 'develop' on remote, that base 'origin/main'.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (main)
$ git checkout -b develop origin/main
Switched to a new branch 'develop'
branch 'develop' set up to track 'origin/main'.

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (develop)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
```

```
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/vspauto/demo-repo/pull/new/develop
remote:
To https://github.com/vspauto/demo-repo.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

$ git ls-remote origin
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        HEAD
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/develop
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/main

$ git branch
* develop
  main
```

We can see new 'develop' branch is created on Github.



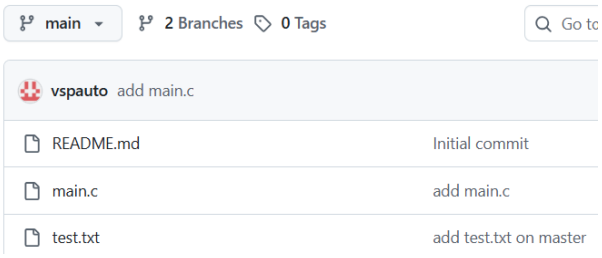
Upload on Branch

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (develop)
$ echo "create file on develop brnach" > develop.txt

$ git add develop.txt

$ git commit -m 'add develop.txt'
[develop 34423de] add develop.txt
 1 file changed, 1 insertion(+)
 create mode 100644 develop.txt

$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 374 bytes | 187.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/vspauto/demo-repo.git
   f9f3f47..34423de  develop -> develop
```

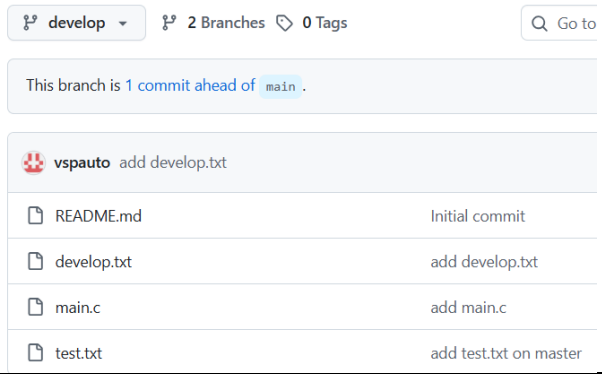
```
$ git ls-remote origin
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        HEAD
34423de4089ae4067bf51e4aa96c2eaa76cb757a        refs/heads/develop
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/main
```

Github has

main branch



develop branch



On '~/demo-repo', sync and then compare two branches.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
$ git fetch origin main
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD

$ git merge
Already up to date.

$ ls
README.md  main.c  test.txt
```

For develop branch,

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
$ git checkout -b develop
Switched to a new branch 'develop'

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (develop)
$ git fetch origin develop
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 354 bytes | 13.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            develop    -> FETCH_HEAD
 * [new branch]      develop    -> origin/develop

$ git merge origin/develop
Updating f9f3f47..34423de
Fast-forward
 develop.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 develop.txt

$ ls
README.md  develop.txt  main.c  test.txt
```

The 'develop' branch has new develop.txt.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (develop)
$ git log -5 --oneline --graph
* 34423de (HEAD -> develop, origin/develop) add develop.txt
* f9f3f47 (origin/main, origin/HEAD, main) add main.c
*   ccdb759 Merge branch 'main' of https://github.com/vspauto/demo-repo
|\
| * 196d873 remove local.txt test.txt
| *   4e98a93 Merge branch 'main' of https://github.com/vspauto/demo-repo
```

```
| |\
```

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (develop)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
$ git log -5 --oneline --graph
* f9f3f47 (HEAD -> main, origin/main, origin/HEAD) add main.c
*   ccdb759 Merge branch 'main' of https://github.com/vspauto/demo-repo
|\
| * 196d873 remove local.txt test.txt
| *   4e98a93 Merge branch 'main' of https://github.com/vspauto/demo-repo
| |\
| | * b6354aa Initial commit
```

Delete Branch

To delete branch on remote repository, use “git push origin --delete <branch>”.
And then delete it on local repository.

At first, I will create ‘temp’ branch from origin/main. And then push to remote.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (develop)
$ git checkout -b temp origin/main
Switched to a new branch 'temp'
branch 'temp' set up to track 'origin/main'.

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (temp)
$ git push -u origin temp
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'temp' on GitHub by visiting:
remote:      https://github.com/vspauto/demo-repo/pull/new/temp
remote:
To https://github.com/vspauto/demo-repo.git
 * [new branch]      temp -> temp
branch 'temp' set up to track 'origin/temp'.

$ git ls-remote origin
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        HEAD
34423de4089ae4067bf51e4aa96c2eaa76cb757a        refs/heads/develop
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/main
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/temp
```

Delete ‘temp’ on remote.

```
$ git push origin --delete temp
To https://github.com/vspauto/demo-repo.git
 - [deleted]         temp
```

After you delete it on GitHub, the branch will still exist on your local repository. Delete ‘temp’ on local repository.

```
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo2 (main)
$ git branch -d temp
Deleted branch temp (was f9f3f47).

$ git ls-remote origin
```

```
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        HEAD
34423de4089ae4067bf51e4aa96c2eaa76cb757a        refs/heads/develop
f9f3f470eb05d5ceb353ec6b75f97fdec3812b12        refs/heads/main
```

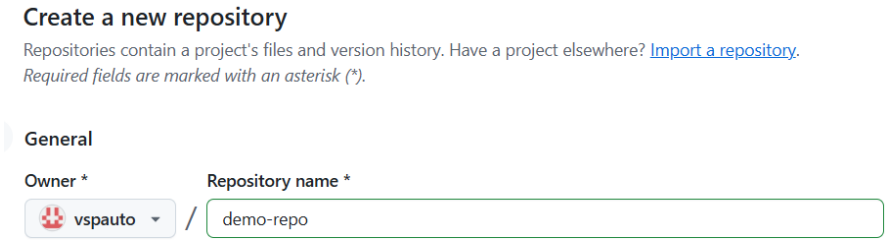
6 Github

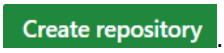
6.1 Account Setup and Configuration

Visit https://github.com, sign-up.

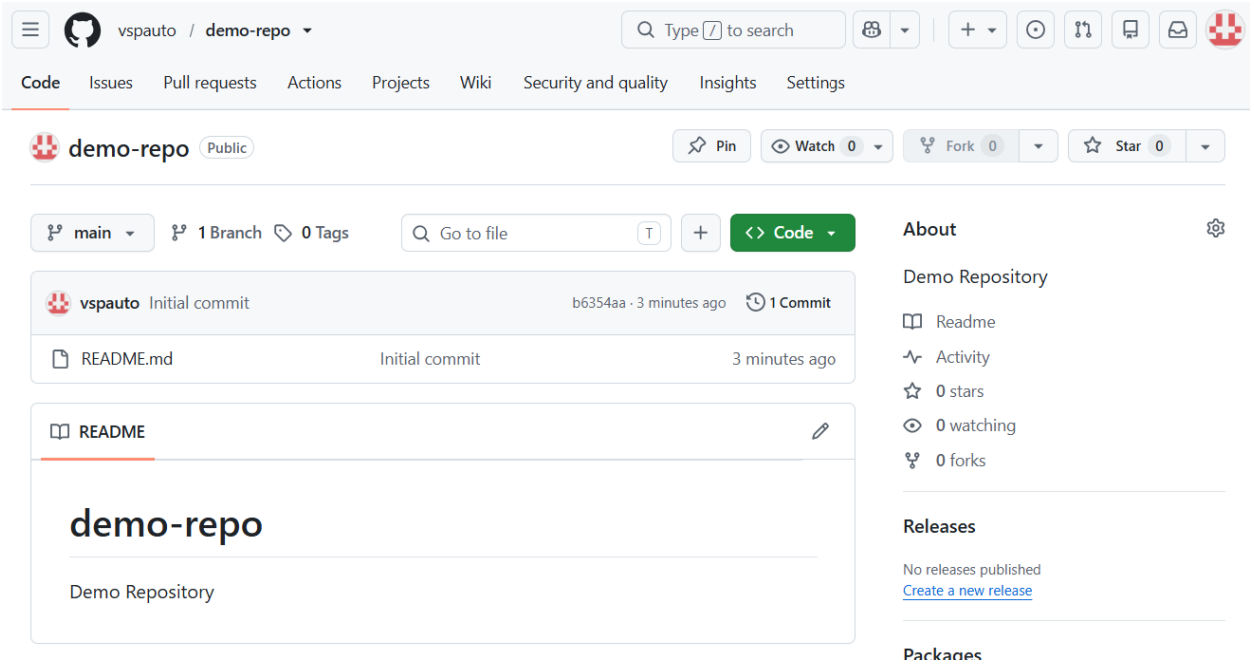
6.2 Create new repository

Select  on top/left, and select new repository.

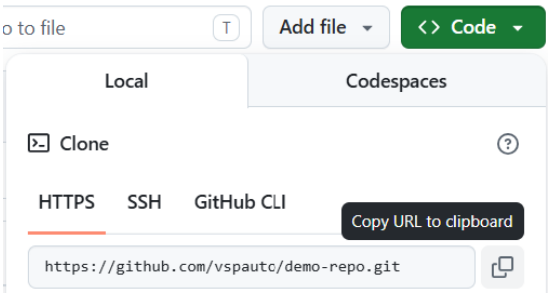


Select below items, and then click  .

Item	
Owner	In my case, vspauto
Repository name	‘demo-repo’
Description	It will appear on ‘github.com/vspauto’, and used on README.md
visibility	Select public
Add README	Select ON
.gitignore	No .gitignore. We will change it later
License	Select No License



select "**<> code**" and copy <url>. In this case, it is 'https://github.com/vspauto/demo-repo.git'.



6.3 Prepare synched local repository

Below command will generate project directory on "~/demo-repo".

```
$ cd ~
$ git clone https://github.com/vspauto/demo-repo.git
Cloning into 'demo-repo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

bongh@DESKTOP-INN3QB3 MINGW64 ~
$ cd demo-repo

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
```

Below command also will generate project directory on "~/demo-repo1".

```
$ cd ~
$ mkdir demo-repo1
$ cd demo-repo1
```

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1
$ git init
Initialized empty Git repository in C:/Users/bongh/demo-repo1/.git/
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (master)
$ git branch -m master main
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (main)
$ git remote add origin https://github.com/vspauto/demo-repo.git
$ git pull origin main
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 20 (delta 0), reused 18 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (20/20), 2.60 KiB | 11.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
```

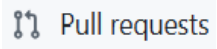
In this case, we rename ‘master’ to ‘main’ branch, add remote on ‘origin’, and then pull remote to local repository. Do not execute ‘git commit’ command before you execute first ‘git pull’ command.

If remote has a branch, create local branch, and then execute ‘git pull’ command.

```
bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (main)
$ git branch
* main

$ git checkout -b develop
Switched to a new branch 'develop'

bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo1 (develop)
$ ls
README.md  main.c  test.txt

$ git pull origin develop
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 354 bytes | 7.00 KiB/s, done.
From https://github.com/vspauto/demo-repo
 * branch            develop    -> FETCH_HEAD
 * [new branch]      develop    -> origin/develop
Updating f9f3f47..34423de
Fast-forward
 develop.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 develop.txt

$ ls
README.md  develop.txt  main.c  test.txt

$ git branch
* develop
  main
```

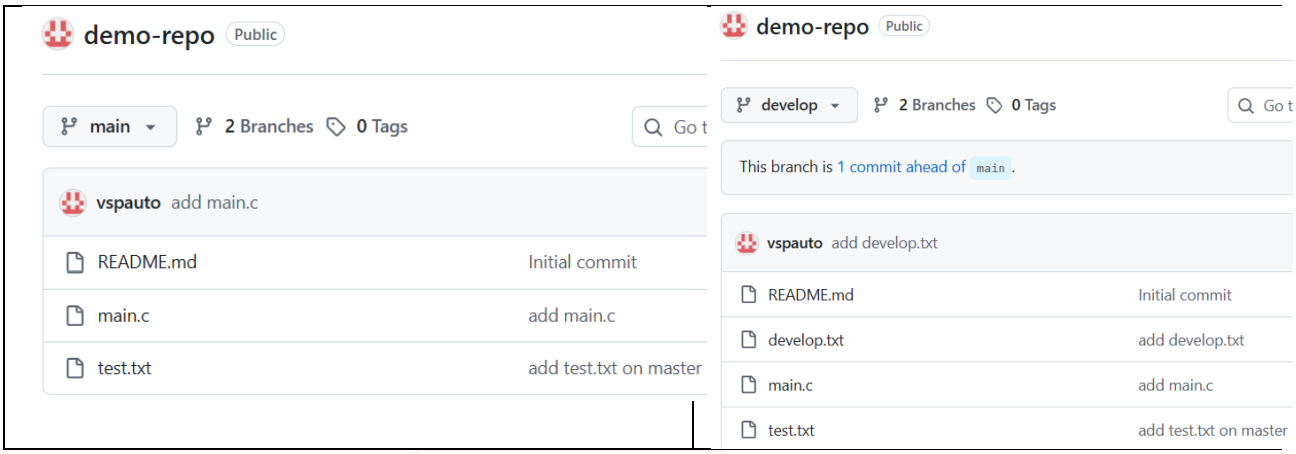
From now on, we can use commands we learned from previous chapters. Create or modify file, commit to local repository, and then push to remote.

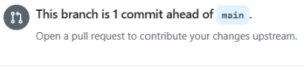
6.3 Pull Request

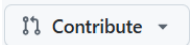


On the below environment, we want to merge develop branch on main branch.

main	develop

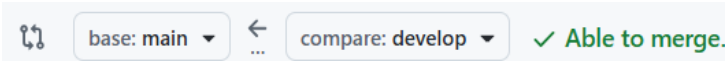




Select 

. It shows 

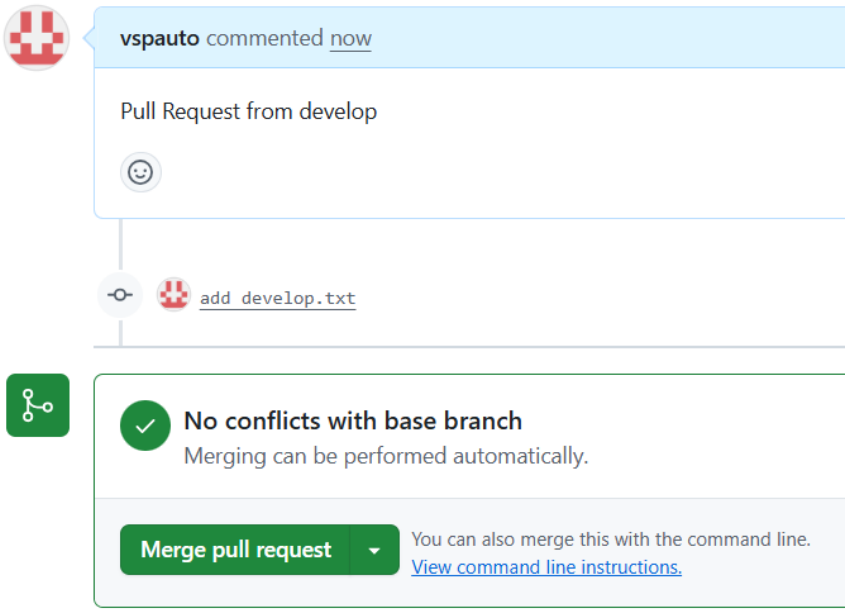
dialog box. Select ‘Open pull request”.
On the top area, it shows the direction (from develop to main), and “able to merge”.



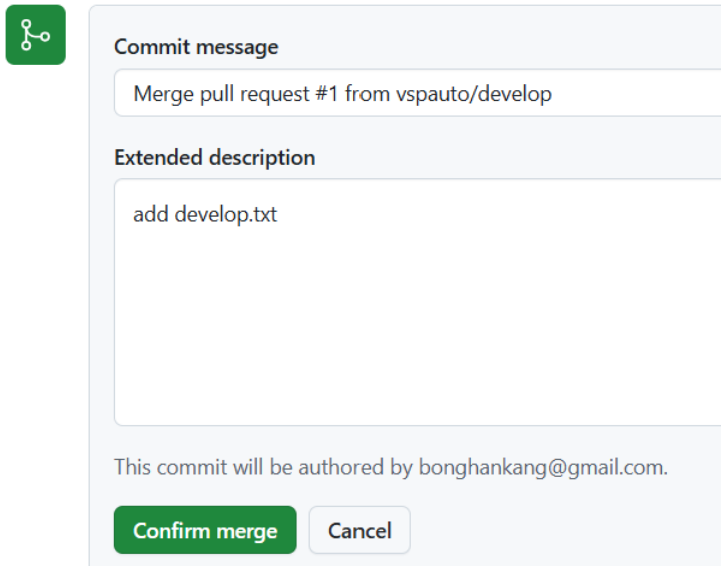
Select 

with “Pull Request from develop” message.

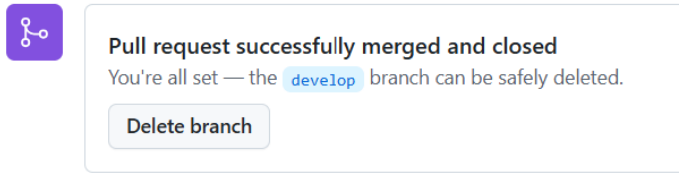
Github draws below screen,



Select “Merge pull request”.
It provides dialog box to add commit message. Select “Confirm merge”.



Done, Github provides below message. I want to keep develop branch, so all done.



If you want to confirm the merge on local, executes ‘git fetch’ and ‘git merge’ commands.

before	$ git checkout main
	bongh@DESKTOP-INN3QB3 MINGW64 ~/demo-repo (main)
	$ git log --oneline --graph
	* f9f3f47 (HEAD -> main, origin/main, origin/HEAD) add main.c
	* ccdb759 Merge branch 'main' of https://github.com/vspauto/demo-repo
	|\
	| * 196d873 remove local.txt test.txt
	| * 4e98a93 Merge branch 'main' of https://github.com/vspauto/demo-repo
	| |\
	| | * b6354aa Initial commit
	| * c19c63d local
	| * e7d22fc add test.txt
	* 09507b0 add test.txt on master

sync	$ git fetch origin main
	remote: Enumerating objects: 1, done.
	remote: Counting objects: 100% (1/1), done.
	remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
	Unpacking objects: 100% (1/1), 889 bytes | 63.00 KiB/s, done.
	From https://github.com/vspauto/demo-repo
	* branch main -> FETCH_HEAD
	f9f3f47..ddaddf4 main -> origin/main
	
	$ git merge
	Updating f9f3f47..ddaddf4
	Fast-forward
	develop.txt | 1 +
	1 file changed, 1 insertion(+)
	create mode 100644 develop.txt

after	$ git log --oneline --graph * ddaddf4 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from vspauto/develop |\

```
| * 34423de (origin/develop, develop) add develop.txt
|/
* f9f3f47 add main.c
*   ccdb759 Merge branch 'main' of https://github.com/vspauto/demo-repo
|\
| * 196d873 remove local.txt test.txt
| *   4e98a93 Merge branch 'main' of https://github.com/vspauto/demo-repo
| |\
| | * b6354aa Initial commit
| * c19c63d local
| * e7d22fc add test.txt
* 09507b0 add test.txt on master
```

Github workflow.

1	Clone the repository
2	Create a new branch from the main (or another branch)
3	Make your changes
4	Push the branch to the remote repository.
5	Open a Pull Request
6	Merge the changes
7	Pull the merged changes into your local main branch
8	Repeat from step2.

6.4 Resolving conflict merge.

We learned how to solve the conflict merge on previous chapter, with CLI. We can use same method on Github environment.

From John	From Adam
$ git branch dev-john	

	$ git checkout –b dev-adam
	$ vi main.c
	`int main(int argc, char *argv[])`
	`{`
	`  printf(“Hello from adam\n”);`
	`}`
	$ git add .
	$ git commit –m “Add from adam”
	$ git push –u origin dev-adam

	Create Pull Request to main. On main, Request merge request, and confirm message.

$ git checkout dev-john
$ vi main.c
`int main(int argc, char *argv[])`
`{`
`  printf(“Hello from john\n”);`
`}`
$ git add .
$ git commit –m “Add from john”
$ git push –u origin dev-john	

Create Pull Request	

In this case, Github will show you conflict.

To solve, merge ‘main’ to your ‘dev-john’ branch.

```
$ git checkout main
$ git pull origin main
$ git checkout dev-john
$ git merge main
```

This time, Git find conflict on main.c and refuse to merge. It needs to manually merge to solve the conflict that we learned before.

$ vi main.c

before	`int main(int argc, char *argv[])`
	`{`
	`<<<< HEAD`
	`    printf(“Hello from john\n”);`
	`=====`
	`    printf(“Hello from adam\n”);`
	`>>>> main`
	`}`

merge	`int main(int argc, char *argv[])`
	`{`
	`    printf(“Hello from adam and john\n”);`
	`}`

Add to stage area and local repository, and then push to origin/main.

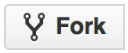
```
$ git add .
$ git commit –m ‘resolve merge conflict’
$ git push
```

Create ‘Pull Request’ again.

6.2 Contributing to a Project

If you are using someone’s project, you do not have write permission. Make a fork, and then make a pu;; request for your modification.

Forking Projects



When you ″fork″ a project, GitHub will make a copy of the project that is entirely yours; it lives in your namespace, and you can push to it.

Note: Fork change it yours, and it has entire commit histories. And it remember where it is forked, so, you can make a ‘Sync Fork’ that get the new commits, or you can request to apply your modification through ‘Pull Request’.
Note: If you want to use CLI, use ‘gh' tool.

1. Install gh, for example ‘brew install gh'
2. Login, “gh auth login”
3. Fork, “gh repo original_account/repository_name

The GitHub Flow

GitHub is designed around a particular collaboration workflow, centered on Pull Requests.
General work flow:

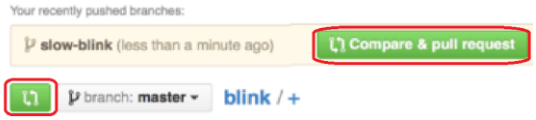
1. Fork the project.
2. Create a topic branch from `main`.
3. Make some commits to improve the project.
4. **Push** this branch to your GitHub project.
5. Open a **Pull Request** on GitHub.
6. Discuss, and optionally continue committing.
7. The project owner merges or closes the Pull Request.
8. **Sync** the updated `main` back to your fork.

Generating a Pull Request

```
bongh@DESKTOP-INN3QB3 MINGW64 /e/github
$ git clone https://github.com/vspauto/git-test.git
Cloning into 'git-test'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.

$ cd git-test

bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (main)
$ ls -al
total 10
drwxr-xr-x 1 bongh 197609  0 Jul  5 22:20 ./
drwxr-xr-x 1 bongh 197609  0 Jul  5 22:20 ../
drwxr-xr-x 1 bongh 197609  0 Jul  5 22:20 .git/
-rw-r--r-- 1 bongh 197609 33 Jul  5 22:20 README.md
-rw-r--r-- 1 bongh 197609 14 Jul  5 22:20 a.txt
```

```
$ cat a.txt
Hello World!

$ git checkout -b branchA
Switched to a new branch 'branchA'

bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (branchA)
$ echo "Add at branchA" >> a.txt

$ git diff

$ git commit -am 'Change a.txt at branchA'
[branchA 63ab9a8] Change a.txt at branchA
 1 file changed, 1 insertion(+)

$ git push origin branchA
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 155.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'branchA' on GitHub by visiting:
remote:      https://github.com/vspauto/git-test/pull/new/branchA
remote:
To https://github.com/vspauto/git-test.git
 * [new branch]      branchA -> branchA
```

On last command, we do push our new topic branch back up to our GitHub fork.



Once the maintainer makes this comment, the person who opened the Pull Request will get a notification. In Pull Request final you can also see that the old code comment has been collapsed in the updated Pull Request, since it was made on a line that has since been changed.

GitHub checks to see if the Pull Request merges cleanly and provides a button to do the merge for you on the server.

This button only shows up if you have write access to the repository and a trivial merge is possible. If you click it GitHub will perform a "non-fast-forward" merge, meaning that even if the merge could be a fast-forward, it will still create a merge commit.

This is the basic workflow that most GitHub projects use. Topic branches are created, Pull Requests are opened on them, a discussion ensues, possibly more work is done on the branch and eventually the request is either closed or merged.

```
bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (branchA)
$ git branch
* branchA
  main

$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (main)
$ cat a.txt
Hello World!
```

```
$ git fetch origin
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 894 bytes | 3.00 KiB/s, done.
From https://github.com/vspauto/git-test
   0a00abd..bac94c2  main       -> origin/main

$ git merge
Updating 0a00abd..bac94c2
Fast-forward
 a.txt | 1 +
 1 file changed, 1 insertion(+)

$ cat a.txt
Hello World!
Add at branchA
```

```
bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (main)
$ git branch
  branchA
* main
```

```
$ git branch -d branchA
Deleted branch branchA (was 63ab9a8).

$ git ls-remote
From https://github.com/vspauto/git-test.git
bac94c28e00778eb6723d8ad3fbf40319937d714        HEAD
63ab9a81be5aebfa73531e0e739bafb26e4dee56        refs/heads/branchA
bac94c28e00778eb6723d8ad3fbf40319937d714        refs/heads/main
63ab9a81be5aebfa73531e0e739bafb26e4dee56        refs/pull/1/head
```

Delete branchA on Github.

```
bongh@DESKTOP-INN3QB3 MINGW64 /e/github/git-test (main)
$ git push origin --delete branchA
To https://github.com/vspauto/git-test.git
 - [deleted]         branchA
```

```
$ git log --oneline --decorate --graph --all
*   bac94c2 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from
vspauto/branchA
|\
| * 63ab9a8 Change a.txt at branchA
|/
* 0a00abd Second commit: Hello World!
* 7b41f6d First commit: Hello WOrld!
* 85008bb Initial commit
```

Keeping up with Upstream

If your Pull Request becomes out of date or otherwise doesn't merge cleanly, you will want to fix it. If you see something like Pull Request does not merge cleanly, you'll want to fix your branch.


```
$ git remote add upstream https://github.com/schacon/blink
$ git fetch upstream
$ git merge upstream/main
$ vim blink.ino
$ git add blink.ino
$ git commit
$ git push origin slow-blink
```

Fix the conflict that occurred on upstream. Push back up to the same topic branch. Once you do that, the Pull Request will be automatically updated and re-checked to see if it merges cleanly.

6.3 Maintaining a Project

Adding Collaborators

If you're working with other people who you want to give commit access to, you need to add them as "collaborators".

Setting -> Collaborators -> and click "Add collaborator."

Managing Pull Requests

Someone send you a Pull Request, you should get an email notifying. It provides some suggested method.

You can merge this Pull Request by running

```
git pull https://github.com/schacon/fade patch-1
```

Or view, comment on, or merge it at:

https://github.com/tonychacon/fade/pull/1

Commit Summary

- wait longer to see the dimming effect better

File Changes

- **M** fade.ino (2)

Patch Links:

- https://github.com/tonychacon/fade/pull/1.patch
- https://github.com/tonychacon/fade/pull/1.diff

a simple way to merge in a remote branch without having to add a remote.

```
git push <url> patch-1
```

If you wish, you can create and switch to a topic branch and then run this command to merge in the Pull Request changes.

The `.diff` and `.patch` URL, provide unified diff and patch versions of the Pull Request. You could technically merge in the Pull Request work with something like this:

```
$ curl https://github.com/tonychacon/fade/pull/1.patch | git am
```

Pull Request Refs

If you're dealing with a **lot** of Pull Requests and don't want to add a bunch of remotes or do one time pulls every time, there is a neat trick that GitHub allows you to do.

```
$ git ls-remote origin
10d539600d86723087810ec636870a504f4fee4d HEAD
10d539600d86723087810ec636870a504f4fee4d refs/heads/master
6a83107c62950be9453aac297bb0193fd743cd6e refs/pull/1/head
afe83c2d1a70674c9505cc1d8b7d380d5e076ed3 refs/pull/1/merge
3c8d735ee16296c242be7a9742ebfbc2665adec1 refs/pull/2/head
15c9f4f80973a2758462ab2066b6ad9fe8dcf03d refs/pull/2/merge
```

```
$ git fetch origin refs/pull/2/head
From https://github.com/libgit2/libgit2
* branch refs/pull/2/head -> FETCH_HEAD
```

This tells Git, "Connect to the **origin** remote, and download the ref named **refs/pull/2/head**." and puts a pointer to the commit you want under **.git/FETCH_HEAD**. You can follow that up with **git merge FETCH_HEAD** into a branch you want to test it in.

6.4 Management an organization

Teams
Organizations are associated with individual people by way of teams, which are simply a grouping of individual user accounts and repositories within the organization and what kind of access those people have in those repositories.

7 Custom config

7.1 Alias

```
$ git config --global alias.co checkout
$ git config --global alias.br branch
$ git config --global alias.ci commit
$ git config --global alias.st status
```

```
$ git config --global alias.unstage 'reset HEAD --'
```

```
$ git config --global alias.last 'log -1 HEAD'
```

```
$ git config --global alias.visual '!gitk'
```

8 Git Cheat Sheet

https://education.github.com/git-cheat-sheet-education.pdf

Installation

GitHub for Windows	https://windows.github.com
GitHub for Mac	https://mac.github.com
For Linux and Solaris platforms	the latest release is available on the official Git web site.
Git for All Platforms	http://git-scm.com

Setup

git config --global user.name “<first lastname>”	set a name that is identifiable for credit when review version history
git config --global user.email “<valid-email>”	set an email address that will be associated with each history marker
git config --global color.ui auto	set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

git init	initialize an existing directory as a Git repository
git clone <url>	retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

git status	show modified files in working directory, staged for your next commit
git add <file>	add a file as it looks now to your next commit (stage)
git reset <file>	unstage a file while retaining the changes in working directory
git diff	diff of what is changed but not staged
git diff --staged	diff of what is staged but not yet committed
git commit -m “<message>”	commit your staged content as a new commit snapshot

Note: Use ‘git restore --staged <file>’ instead of ‘git reset <file>.’

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

git branch	list your branches. a * will appear next to the currently active branch
git branch <branch>	create a new branch at the current commit
git checkout <branch>	switch to another branch and check it out into your working directory
git merge <branch>	merge the specified branch's history into the current one
git log	show all commits in the current branch's history

INSPECT & COMPARE

Examining logs, diffs and object information

git log	show the commit history for the currently active branch
git log branchB..branchA	show the commits on branchA that are not on branchB
git log --follow <file>	show the commits that changed file, even across renames
git diff branchB...branchA	show the diff of what is in branchA that is not in branchB
git show <SHA>	show any object in Git in human-readable format

TRACKING PATH CHANGES

Versioning file removes and path changes

git rm <file>	delete the file from project and stage the removal for commit
git mv <existing-path> <new-path>	change an existing file path and stage the move
git log --stat -M	show all commit logs with indication of any paths that moved

IGNORING PATTERNS

Preventing unintentional staging or committing of files

logs/ *.notes pattern*/	Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.
git config --global core.excludesfile <file>	system wide ignore pattern for all local repositories

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

git remote add <alias> <url>	add a git URL as an alias
git fetch <alias>	fetch down all the branches from that Git remote
git merge <alias>/<branch>	merge a remote branch into your current branch to bring it up to date
git push <alias> <branch>	Transmit local branch commits to the remote repository branch
git pull	fetch and merge any commits from the tracking remote branch

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

git rebase <branch>	apply any commits of current branch ahead of specified one
git reset --hard <commit>	clear staging area, rewrite working tree from specified commit

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

git stash	Save modified and staged changes
git stash list	list stack-order of stashed file changes
git stash pop	write working from top of stash stack

git stash drop	discard the changes from top of stash stack